DIGITAL LOGIC DESIGN

OBJECTIVES

This course provides in-depth knowledge of switching theory and the logic design techniques of digital circuits, which is the basis for design of any digital circuit. The course objectives are:

To learn basic techniques for the design of digital circuits and fundamental concepts used in the design of digital systems.

To understand common forms of number representation in digital electronic circuits and to be able to convert between different representations.

To implement simple logical operations using combinational logic circuits

To design combinational logic circuits, sequential logic circuits.

To impart to student the concepts of sequential circuits, enabling them to analyze sequential systems in terms of state machines.

To implement synchronous state machines using flip-flops.

DLD-Digital Logic Design

A system in electrical and computer engineering that uses simple numerical values to produce input and output operations.

Digital Logic Design involves the study of digital circuits that operate using logic gates to process binary information (0s and 1s).

Digital systems and Binary Numbers

- **Digital:**

Concerned with the interconnection among digital components and modules.

Logic Gates, Flip-Flops, Multiplexers (MUX), Encoders, Counters.

Arithmetic Logic Unit (ALU), Memory Modules, Programmable Logic Devices, Microcontroller and Microprocessor, Digital Display Modules, Communication Modules.

Best digital system Example: General Purpose computer.

- **Logic Design:**

Deals with basic concepts and tools used to design digital hardware consisting of logic circuits.

Circuit to perform arithmetic operations.

NUMBER SYSTEMS & BOOLEAN ALGEBRA

- Introduction about digital system
- Philosophy of number systems
- Complement representation of negative numbers
- Binary arithmetic
- Binary codes
- Error detecting & error correcting codes
- Hamming codes

INTRODUCTION ABOUT DIGITAL SYSTEM

A Digital system is an interconnection of digital modules

A system that manipulates discrete elements of information that is represented internally in the binary form.

- Used in industrial and consumer products such as automated industrial machinery, pocket calculators, microprocessors, digital computers, digital watches, TV games and signal processing and so on.

Characteristics of Digital systems

- Digital systems manipulate discrete elements of information.
- Discrete elements are nothing but the digits such as 10 decimal digits or 26 letters of alphabets and so on.
- Digital systems use physical quantities called signals to represent discrete elements.
- In digital systems, the signals have two discrete values and are therefore said to be binary.
- A signal in digital system represents one binary digit called a bit. The bit has a value either 0 or 1.

Analog systems vs Digital systems

- Analog system process information that varies continuously
- Process time varying signals that can take on any values across a continuous range of voltage, current or any physical parameter.

- Digital systems use digital circuits that can process digital signals which can take either 0 or 1 for binary system.

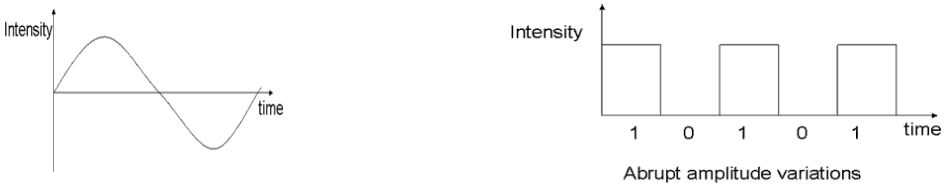


Figure 1

NUMBER SYSTEM:

- Number system is a basis for counting varies items.
- Modern computers communicate and operate with binary numbers which use only the digits 0 &1.
- Basic number system used by humans is Decimal number system.

For Ex: Let us consider decimal number 18. This number is represented in binary as 10010.

- We observe that binary number system take more digits to represent the decimal number.
- For large numbers we have to deal with very large binary strings. So this fact gave rise to three new number systems

Octal number systems
Hexa Decimal number system
Binary Coded Decimal number (BCD) system

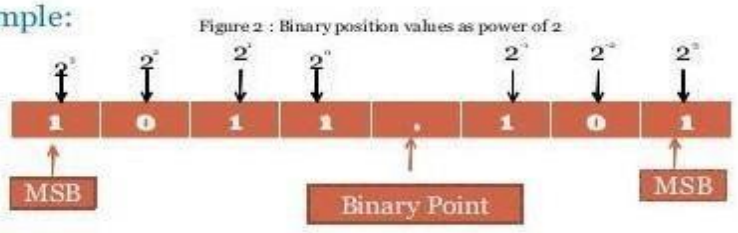
To define any number system we have to specify

- Base of the number system such as 2,8,10 or 16.
- The base decides the total number of digits available in that number system.
- First digit in the number system is always zero and last digit in the number system is always base-1.

Binary number system:

The binary number has a radix of 2. As r = 2, only two digits are needed, and these are 0 and 1. In binary system weight is expressed as power of 2.

- Example:



Decimal Number system

The decimal system has ten symbols: 0,1,2,3,4,5,6,7,8,9. In other words, it has a base of 10.

COMPLEMENTS

Complements are used in the digital computers in order to simplify the subtraction operation and for the logical manipulations.
For each radix-r system (radix r represents base of number system)
There are two types of complements.

R's complement/Radix

(R-1)'s complement/Diminished Radix

Radix Complement and Diminished Radix Complement

The mostly used complements are 1's, 2's, 9's, and 10's complement. Apart from these complements, there are many more complements.

For finding the subtraction of the number base system, the complements are used.

If **r** is the base of the number system, then there are two types of complements that are possible, i.e. r's and (r-1)'s. We can find the r's complement, and (r-1)'s complement of the number, here r is the radix. The r's complement is also known as **Radix complement** (r-1)'s complement, is known as **Diminished Radix complement**.

If the base of the number is 2, then we can find 1's and 2's complement of the number. Similarly, if the number is the octal number, then we can find 7's and 8's complement of the number.

formula for finding the r's and (r-1)'s complement:

$$r's = complement = (r^n)_{10} - N$$

$$(r-1)'s\ complement = \{(r^n)_{10} - 1\} - N$$

In the above formulas,
The n is the number of digits in the number.

The N is the given number.

The r is the radix or base of the number.

Let's take some examples to understand how we can calculate the r's and (r-1)'s complement of binary, decimal, octal, and hexadecimal numbers.

Example 1: $(1011000)_2$

This number has a base of 2, which means it is a binary number. So, for the binary numbers, the value of r is 2, and r-1 is 2-1=1. So, we can calculate the 1's and 2's complement of the number.

1's complement of the number 1011000 is calculated as:

$=\{(2^7)_{10}-1\}-(1011000)_2$

$=\{(128)_{10}-1\}-(1011000)_2$

$=\{(127)_{10}\}-(1011000)_2$

$=1111111_2-1011000_2$

$=0100111$

2's complement of the number 1011000 is calculated as:

$=(2^7)_{10}-(1011000)_2$

$=(128)_{10}-(1011000)_2$

$=10000000_2-1011000_2$

$=0101000_2$

Example 2: $(155)_{10}$

This number has a base of 10, which means it is a decimal number. So, for the decimal numbers, the value of r is 10, and r-1 is 10-1=9. So, we can calculate the 10's and 9's complement of the number.

9's complement of the number 155 is calculated as:

$=\{(10^3)_{10}-1\}-(155)_{10}$

$=(1000-1)-155$

$=999-155$

$=(844)_{10}$

10's complement of the number 155 is calculated as:

$=(10^3)_{10}-(155)_{10}$

$=1000-155$

$=(845)_{10}$

Example 3: $(172)_8$

This number has a base of 8, which means it is an octal number. So, for the octal numbers, the value of r is 8, and r-1 is 8-1=7. So, we can calculate the 8's and 7's complement of the number.

7's complement of the number 172 is calculated as:

$=\{(8^3)_{10}-1\}-(172)_8$

$=((512)_{10}-1)-(122)_8$

$=(511)_{10}-(122)_{10}$

$=(389)_{10}$

$=(605)_8$

8's complement of the number 172 is calculated as:

$=(8^3)_{10}-(172)_8$

$=(512)_{10}-172_8$

$=512_{10}-122_{10}$

$=390_{10}$

$=606_8$

Example 4: $(F9)_{16}$

This number has a base of 16, which means it is a hexadecimal number. So, for the hexadecimal numbers, the value of r is 16, and r-1 is 16-1=15. So, we can calculate the 16's and 15's complement of the number.

15's complement of the number F9 is calculated as:

$\{(16^2)_{10}-1\}$

$(F9)_{16}\ (256-1)_{10}-$

$F9_{16}\ 255_{10}-249_{10}$

$(6)_{10}$

$(6)_{16}$

16's complement of the number F9 is calculated as:

$\{(16^2)_{10}\}-(F9)_{16}$

$256_{10}-249_{10}$

$(7)_{10}$

$(7)_{16}$

r's complement

The r's complement of a non-zero number in any number system with base r can be calculated by adding 1 to the LSB of its (r-1)'s complement.

For Example: In binary number system, **2's** complement of **001** can be calculated by adding 1 to the LSB of its 1'complement (i.e., **110 + 1**)

= $(111)_2$.

Similarly, in octal number system, **8's** complement of **347** can be calculated by adding 1 to the LSB of its **7'complement** (i.e., **430 + 1**) = **$(431)_8$**.

(r-1)'s complement

The **(r-1)'s complement of a number** in any number system with base **r** can be found out by subtracting every single digit of a number by **r-1**.

For Example: In the binary number system, the base is **2**. Hence, its **(r- 1)'s** i.e., **(2-1 =1)'s** complement can be obtained by subtracting each bit from 1, i.e., **1's** complement for **001** can also be calculated by

Subtracting **001** from **111** which will be **(111-001) = $(110)_2$**.

Similarly, in the octal number system, the base is **8** so its **7's** complement can be calculated by subtracting each bit by **7**, i.e., **7's** complement for **347** in octal number system can be calculated by subtracting **347** from **777** which will yield **(777 – 347) = $(430)_8$**.

9's and 10's complement in Decimal Number System

We already know that the decimal number system has its base as 10, As we have already discussed above, 9's complement of decimal number can be found out by subtracting its each by 9.

Example 1: Calculate 9's complement of $(2457)^{10}$

Solution: $9999 - 2457 = (7542)^{10}$

Now, 10's complement of a decimal number can be calculated by adding 1 to the LSB of the 9's complement.

Example 2: Calculate 10's complement of $(2457)^{10}$

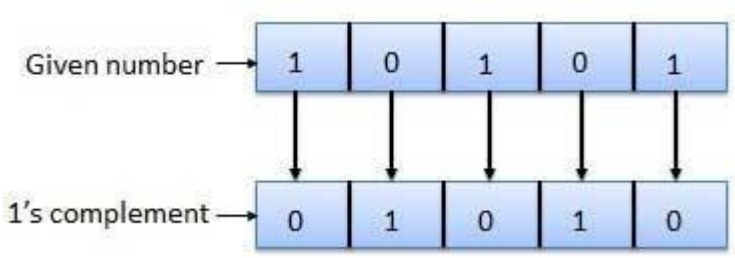
Solution: 10's complement for $(2457)^{10}$ is calculated by adding 1 to $(7542)^{10}$ which is the 9's complement. Therefore, 10's complement of $(2457)^{10}$ is $(7543)^{10}$

Binary system complements

As the binary system has base r = 2. So the two types of complements for the binary system are 2's complement and 1's complement.

1's complement

The 1's complement of a number is found by changing all 1's to 0's and all 0's to 1's. This is called as taking complement or 1's complement. Example of 1's Complement is as follows

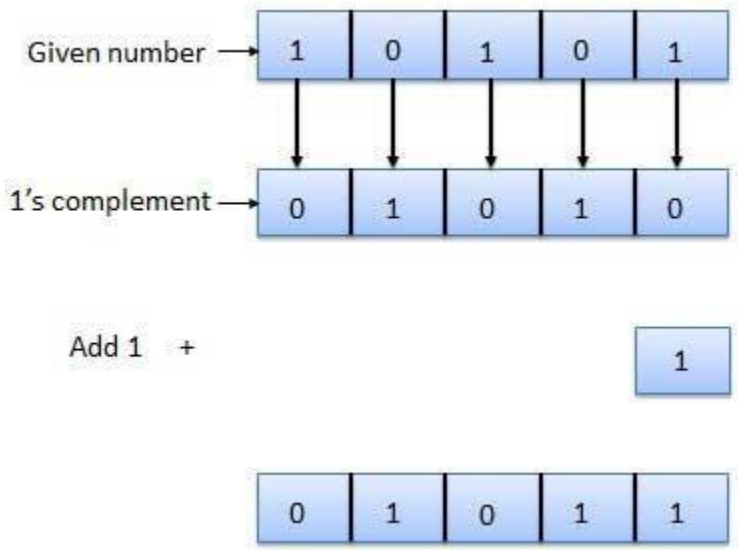


2's complement

The 2's complement of binary number is obtained by adding 1 to the Least Significant Bit (LSB) of 1's complement of the number.

2's complement = 1's complement + 1

Example of 2's Complement is as follows.



Digital Logic Gates:
Boolean functions are expressed in terms of AND, OR, and NOT operations.
it is easier to implement a Boolean function with these type of gates.

Name	Graphic symbol	Algebraic function	Truth table
AND		$F = x \cdot y$	x y | F 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1
OR		$F = x + y$	x y | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1
Inverter		$F = x'$	x | F 0 | 1 1 | 0
Buffer		$F = x$	x | F 0 | 0 1 | 1
NAND		$F = (xy)'$	x y | F 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0
NOR		$F = (x + y)'$	x y | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	x y | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	x y | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1

Universal Logic Gates

NAND and NOR gates are called Universal gates.
All fundamental gates (NOT, AND, OR) can be realized by using either only NAND or only NOR gate.
A universal gate provides flexibility and offers enormous advantage to logic designers.

Boolean Algebra
A system of mathematical logic.
It is an algebraic system consisting of the set of elements (0, 1), two binary operators called OR, AND, and one unary operator NOT.
It is the basic mathematical tool in the analysis and synthesis of switching circuits.
It is set of rules, used to simplify the given logic expression without changing its functionality.

Axioms and laws of Boolean algebra

Axioms or Postulates of Boolean algebra are a set of logical expressions that we accept without proof and upon which we can build a set of useful theorems.

	AND Operation	**OR Operation**	**NOT Operation**
Axiom1 :	0.0=0	0+0=0	$\overline{0}$=1
Axiom2:	0.1=0	0+1=1	$\overline{1}$=0
Axiom3:	1.0=0	1+0=1	
Axiom4:	1.1=1	1+1=1	

AND Law

Law1: A.0=0 (Null law)
Law2: A.1=A (Identity law)
Law3: A.A=A (Impotence law)

OR Law

Law1: A+0=A
Law2: A+1=1
Law3: A+A=A (Impotence law)

ASSOCIATIVE LAW:

A binary operator * on a set *S* is said to be associative whenever $(x * y) * z = x * (y * z)$

COMMUTATIVE LAW:

A binary operator * on a set *S* is said to be commutative whenever $x * y = y * x$

BASIC IDENTITIES OF BOOLEAN ALGEBRA

Postulate 1(Definition): A Boolean algebra is a closed algebraic system containing a set *K* of two or more elements and the two operators · and + which refer to logical AND and logical OR

- $x + 0 = x$
- $x \cdot 0 = 0$
- $x + 1 = 1$
- $x \cdot 1 = 1$
- $x + x = x$
- $x \cdot x = x$
- $x + x' = x$
- $x \cdot x' = 0$
- $x + y = y + x$
- $xy = yx$
- $x + (y + z) = (x + y) + z$
- $x (yz) = (xy) z$
- $x (y + z) = xy + xz$
- $x + yz = (x + y)(x + z)$ **Distributive**

- *(x + y)' = x' y'*
- *(xy)' = x' + y'*
- *(x')' = x*
- *x + x'=1 and x. x'=0*

DeMorgan's Theorem

(a) $(a + b)' = a'b'$

(b) $(ab)' = a' + b'$

Generalized DeMorgan's Theorem

(a) $(a + b + \ldots z)' = a'b' \ldots z'$
(b) $(a.b \ldots z)' = a' + b' + \ldots z'$

Basic Theorems and Properties of Boolean algebra Commutative law

Law1: A+B=B+A Law2: A.B=B.A

Associative law

Law1: A + (B +C) = (A +B) +C Law2: A(B.C) = (A.B)C

Distributive law

Law1: A.(B + C) = AB+ AC Law2: A + BC = (A + B).(A +C)

Absorption law

Law1: A +AB =A

Solution: A(1+B)
A

Law2: A(A +B) = A

Solution: A.A+A.B
A+A.B
A(1+B)
A

Consensus Theorem

The Consensus Theorem is a simplification rule used in Boolean algebra that helps **reduce redundant** terms in logic expressions. It states that:

AB+ A'C + BC = AB + A'C
The term **BC** is redundant because it is already covered by **AB** and **$\bar{A}$C**.
Removing **BC** does not change the logic function.

Proof using Boolean Algebra:
AB+A'C+BC = AB+A'C+(A+A')BC
=AB+A'C+ABC+A'BC
=AB(1+C)+A'C(1+B)
= AB+ A'C

This theorem is useful in simplifying digital logic circuits, minimizing the number of gates required for implementation.

Table for Theorems of Boolean algebra

Part-A	Part-B
A+0=A	A.0=0
A+1=1	A.1=A
A+A=A (Impotence law)	A.A=A (Impotence law)
$A+\bar{A}=1$	$A.\bar{A}=0$
$\bar{\bar{A}}=A$ (double inversion law)	--
Commutative law: A+B=B+A	A.B=B.A
Associative law: A + (B +C) = (A +B) +C	A(B.C) = (A.B)C
Distributive law: A.(B + C) = AB+ AC	A + BC = (A + B).(A +C)
Absorption law: A +AB =A	A(A +B) = A
DeMorgan Theorem: $\overline{(A+B)}=\bar{A}.\bar{B}$	$\overline{(A.B)}=\bar{A}+\bar{B}$
Redundant Literal Rule: $A+\bar{A}B=A+B$	$A.(\bar{A}+B)=AB$
Consensus Theorem: AB+ A'C + BC = AB + A'C	(A+B). (A'+C).(B+C) =(A+B).(A'+C)

Boolean Function

Boolean algebra is an algebra that deals with binary variables and logic operations.
A Boolean function described by an algebraic expression consists of binary variables, the constants 0 and 1, and the logic operation symbols.
For a given value of the binary variables, the function can be equal to either 1 or 0.

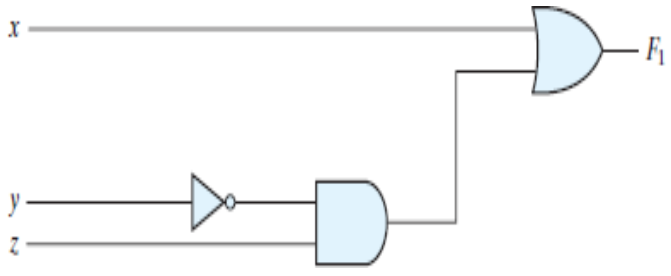
Representation of Boolean Functions

A Boolean function can be represented in multiple ways:

1. **Truth Table**
   A table listing all possible input combinations and their corresponding output.
2. **Boolean Algebra Expression**
   A formula consisting of **AND (·), OR (+), NOT ($\bar{}$)** operations.
   Example: $F(A,B,C)=AB+A\bar{C}$
3. **Logic Circuit**
   A graphical representation using logic gates.
4. **Karnaugh Map (K-Map)**
   A simplification tool to minimize Boolean expressions.

Truth Table for F1=x+ *y'*z

x	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1



Algebraic Manipulation (Minimization of Boolean function)

Boolean algebra is a useful tool for simplifying digital circuits.

Why do it? Simpler can mean cheaper, smaller, faster.

Example: Simplify F = x'yz + x'yz' + xz.

$$F = x'yz + x'yz' + xz$$
$$= x'y(z+z') + xz$$
$$= x'y \cdot 1 + xz$$
$$= x'y + xz$$

Example: Prove

$$x'y'z' + x'yz' + xyz' = x'z' + yz'$$

Complement of a Boolean Function

The **complement** of a Boolean function F(A,B,C,...) denoted as F′ or $\bar{F}$, is obtained by swapping **0s and 1s** in its truth table or applying **De Morgan's Theorem** to its Boolean expression.

Example
Find G(x,y,z)
The complement of F(x,y,z) = xy'z' +x'yz
G = F' = (xy'z' + x'yz)'
= (xy'z')' • (x'yz)' *DeMorgan*
= (x'+y+z) • (x+y'+z') *DeMorgan* again

Canonical and Standard Forms

Canonical and standard forms are **structured ways** to represent **Boolean functions**.
These forms are essential for **logic simplification, circuit design, and minimization techniques.**

1. Canonical Forms

A Boolean function is in canonical form if each term includes **all variables** (either in complemented or uncomplemented form). There are two types:

(a) Canonical Sum of Products (SOP) – Minterm Expansion

It expresses the function as a sum (OR operation) of **minterms** (AND terms where each variable appears in either normal or complemented form).

Each minterm corresponds to a row in the truth table where the **output is 1**.

Notation: $F = \sum m(i)$, where I represents the minterm indices.

Example: Assume 3 variables (A, B, C), and j=3. Then, b_j = 011 and its corresponding minterm is denoted by m_j = A'BC
where j is the decimal equivalent of the minterm's corresponding binary combination (b_j).

A variable in m_j is complemented if its value in b_j is 0, otherwise is uncomplemented

Given the truth table:

A	B	C	F(A, B, C)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

F(A,B,C)=A'B'C+A'BC+AB'C+AB'C

(b) Canonical Product of Sums (POS) – Maxterm Expansion

It expresses the function as a product (AND operation) of **maxterms** (OR terms where each variable appears in either normal or complemented form).

Each maxterm corresponds to a row in the **truth table where the output is 0**.

Notation: F=∏ M(i), where i represents the maxterm indices.

From the previous truth table, the rows where F=0F = 0F=0 are:

F(A,B,C)=(A+B+C)(A+B′+C′)(A′+B+C′)(A′+B′+C′)

F(A,B,C)=∏M(0,2,5,7)

Example: Assume 3 variables (A, B, C), and *j*=3. Then, b_j = 011 and its corresponding maxterm is denoted by M_j = A+B'+C'
Denoted by M_j, where *j* is the decimal equivalent of the maxterm's corresponding binary combination (b_j).

A variable in M_j is complemented if its value in b_j is 1, otherwise is uncomplemented.

2. Standard Forms

Standard SOP and POS are simplified versions of canonical forms, where each term may not necessarily include all variables.

(a) Standard Sum of Products (SOP)

Contains **ANDed terms (products) ORed together**.

Unlike canonical SOP, **each term may not include all variables**.

F(A,B,C)=AB+BC

(b) Standard Product of Sums (POS)

Contains **ORed terms (sums) ANDed together**

F(A,B,C)=(A+B)(B+C)

Key Differences Between Canonical & Standard Forms

Aspect	Canonical SOP/POS	Standard SOP/POS
Variables in each term	**All variables present**	**Some variables may be missing**
Expression complexity	**More complex** (more terms)	**Simpler** (fewer terms)
Use case	**Used for Karnaugh Map (K-map) and circuit implementation**	**Used in simplified logic circuit design**

Canonical SOP = Sum of **minterms** (each term has all variables).

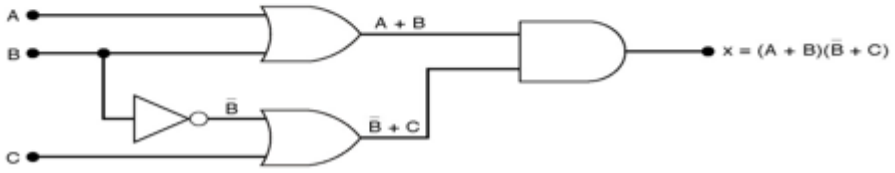
Canonical POS = Product of **maxterms** (each term has all variables).

Standard SOP = Simplified sum of product terms.

Standard POS = Simplified product of sum terms.

Draw the circuit diagram to implement the expression

$$x = (A + B)(\overline{B} + C)$$



Truth table representing minterm and maxterm

			Minterms	Maxterms
X	Y	Z	Product Terms	Sum Terms
0	0	0	$m_0 = \overline{X} \cdot \overline{Y} \cdot \overline{Z} = \min(\overline{X}, \overline{Y}, \overline{Z})$	$M_0 = X + Y + Z = \max(X, Y, Z)$
0	0	1	$m_1 = \overline{X} \cdot \overline{Y} \cdot Z = \min(\overline{X}, \overline{Y}, Z)$	$M_1 = X + Y + \overline{Z} = \max(X, Y, \overline{Z})$
0	1	0	$m_2 = \overline{X} \cdot Y \cdot \overline{Z} = \min(\overline{X}, Y, \overline{Z})$	$M_2 = X + \overline{Y} + Z = \max(X, \overline{Y}, Z)$
0	1	1	$m_3 = \overline{X} \cdot Y \cdot Z = \min(\overline{X}, Y, Z)$	$M_3 = X + \overline{Y} + \overline{Z} = \max(X, \overline{Y}, \overline{Z})$
1	0	0	$m_4 = X \cdot \overline{Y} \cdot \overline{Z} = \min(X, \overline{Y}, \overline{Z})$	$M_4 = \overline{X} + Y + Z = \max(\overline{X}, Y, Z)$
1	0	1	$m_5 = X \cdot \overline{Y} \cdot Z = \min(X, \overline{Y}, Z)$	$M_5 = \overline{X} + Y + \overline{Z} = \max(\overline{X}, Y, \overline{Z})$
1	1	0	$m_6 = X \cdot Y \cdot \overline{Z} = \min(X, Y, \overline{Z})$	$M_6 = \overline{X} + \overline{Y} + Z = \max(\overline{X}, \overline{Y}, Z)$
1	1	1	$m_7 = X \cdot Y \cdot Z = \min(X, Y, Z)$	$M_7 = \overline{X} + \overline{Y} + \overline{Z} = \max(\overline{X}, \overline{Y}, \overline{Z})$

Sum of minterms Example – Express the Boolean function F = A + B''C as standard sum of minterms.
Solution –
A = A(B + B‟) = AB + AB‟
This function is still missing one variable, so
A = AB(C + C‟) + AB''(C + C‟) = ABC + ABC‟+ AB‟C + AB‟C‟

The second term B‟C is missing one variable; hence,
B‟C = B‟C(A + A‟) = AB‟C + A‟B‟C

Combining all terms, we have

F = A + B‟C = ABC + ABC‟ + AB‟C + AB‟C‟ + AB‟C + A‟B‟C

But AB‟C appears twice, and
according to theorem 1 (x + x = x), it is possible to remove one of those occurrences.
Rearranging the minterms in ascending order, we finally obtain

F = A‟B‟C + AB‟C‟ + AB‟C + ABC‟ + ABC
= m1 + m4 + m5 + m6 + m7
SOP is represented as Sigma(1, 4, 5, 6, 7)

Example
Convert the following Boolean function into Standard PoS form.

$$f=(p+q+r)\cdot(p+q+r')\cdot(p+q'+r)\cdot(p'+q+r)$$

The given Boolean function is in canonical PoS, we have to simplify this Boolean function in order to get standard PoS form

$x \cdot x = x$. That means. So, we can write the first term p+q+r two more times.

$$\Rightarrow f=(p+q+r)\cdot(p+q+r)\cdot(p+q+r)\cdot(p+q+r')\cdot(p+q'+r)\cdot(p'+q+r)$$

Step 2 − Use **Distributive law,** $x + (y \cdot z) = (x + y)\cdot(x + z)$ for 1st and 4th parenthesis, 2nd and 5th parenthesis, 3rd and 6th parenthesis.

$$\Rightarrow f=(p+q+rr')\cdot(p+r+qq')\cdot(q+r+pp')$$

Use **Boolean postulate**, x.x'=0 for simplifying the terms present in each parenthesis.

$$\Rightarrow f=(p+q+0)\cdot(p+r+0)\cdot(q+r+0)$$

Use **Boolean postulate**, $x + 0 = x$ for simplifying the terms present in each parenthesis

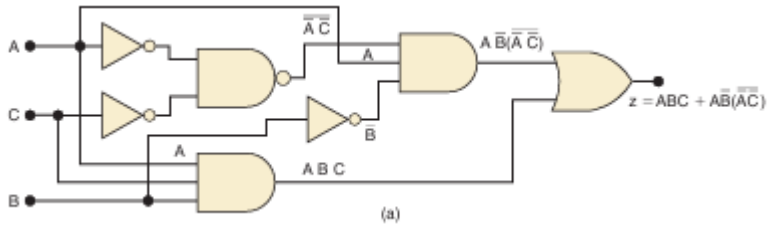
$$\Rightarrow f=(p+q)\cdot(p+r)\cdot(q+r) \Rightarrow f=(p+q)\cdot(p+r)\cdot(q+r)$$

$$\Rightarrow f=(p+q)\cdot(q+r)\cdot(p+r) \Rightarrow f=(p+q)\cdot(q+r)\cdot(p+r)$$

This is the simplified Boolean function. Therefore, the **standard PoS form** corresponding to given canonical PoS form is **f = (p + q)·(q + r)·(p + r)**.

Combination logic circuit and design

Simplify the logic circuit shown in Figure



The first step is to determine the expression for the output

$$z = ABC + A\overline{B} \cdot (\overline{\overline{A}\ \overline{C}})$$

Once the expression is determined, it is usually a to break down all large inverter signs using DeMorgan's theorems and then multiply out all terms.

$$\begin{aligned} z &= ABC + A\overline{B}(\overline{\overline{A}} + \overline{\overline{C}}) && [\text{[redacted]}] \\ &= ABC + A\overline{B}(A + C) && [\text{cancel double inversions}] \\ &= ABC + A\overline{B}A + A\overline{B}C && [\text{multiply out}] \\ &= ABC + A\overline{B} + A\overline{B}C && [A \cdot A = A] \end{aligned}$$

The first and third terms above have AC in common, which can be

$$z = AC(B + \overline{B}) + A\overline{B}$$

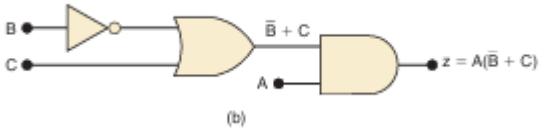
Since $B + \overline{B} = 1$, then

$$\begin{aligned} z &= AC(1) + A\overline{B} \\ &= AC + A\overline{B} \end{aligned}$$

We can now factor out A, which results in

$$z = A(C + \overline{B})$$

This result can be simplified no further. Its circuit implementation is shown in Figure (b). It is obvious that the circuit in Figure (b) is a simpler than the original circuit in Figure (a).

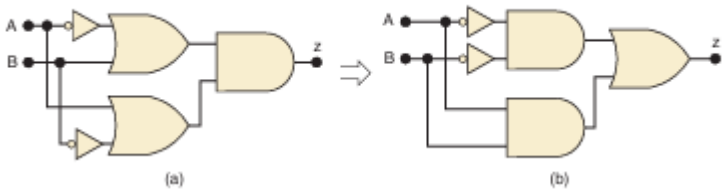


Simplify the expression

$$z = A\overline{B}\ \overline{C} + A\overline{B}C + ABC.$$

Simplify $z = \overline{A}C(\overline{ABD}) + \overline{A}B\overline{C}\,\overline{D} + A\overline{B}C.$

Simplify the circuit of Figure



Simplification Methods (K-Map, Quine-McCluskey Method)

Minimization techniques in logic design are used to simplify Boolean expressions and logic circuits, reducing the number of gates.

1. Algebraic Simplification

Uses Boolean algebra rules (Idempotent Law, De Morgan's Theorems) to simplify expression.

2. Karnaugh Map (K-Map)

A graphical method used for up to 6-variable expressions. Functions by grouping adjacent **1s.**
To reduce the number of terms in a Boolean expression representing a circuit, it is necessary tofindterms tocombine.

3. Quine-McCluskey Method

A tabular method for systematic simplification

Suitable for more than 6 variables

The Quine–McCluskey method has exponential complexity.

Karnaugh Map (K-Map)

There are four possible minterms in the sum-of-products expansion of a Boolean function in the two variables x and y.
A K-map for a Boolean function in these two variables consists of four cells,
Where a 1 is placed in the cell representing a minterm if this minterm is present in the expansion. Cells are said to be adjacent if the minterms that they represent differ in exactly one literal.
For instance, the cell representing x'y is adjacent to the cells representing xy and x' y'. The four cells and the terms that they represent are shown in Figure

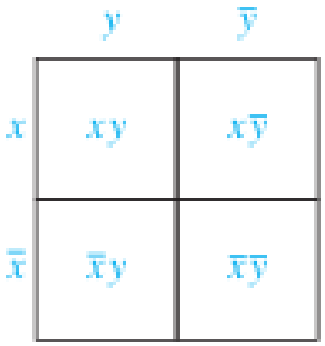
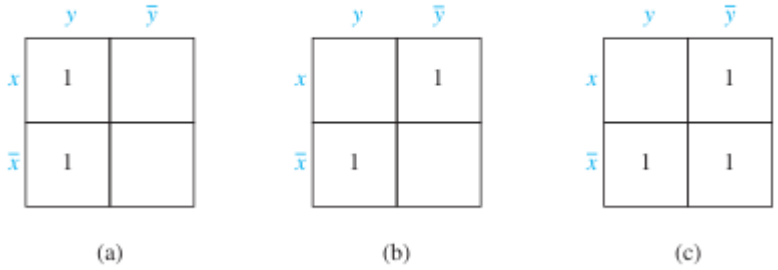


FIGURE
K-maps in Two Variables.

EXAMPLE-1 Find the K-maps for (a) xy + x'y, (b) xy' + x'y, and (c) xy' + x'y + x'y'.

Find the K-maps for (a) $xy + \overline{x}y$, (b) $x\overline{y} + \overline{x}y$, and (c) $x\overline{y} + \overline{x}y + \overline{x}\,\overline{y}$.

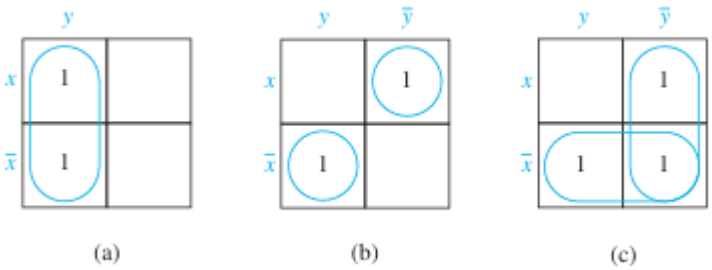
We include a 1 in a cell when the minterm represented by this cell is present in the sum-of-products expansion. The three K-maps are shown in Figure.



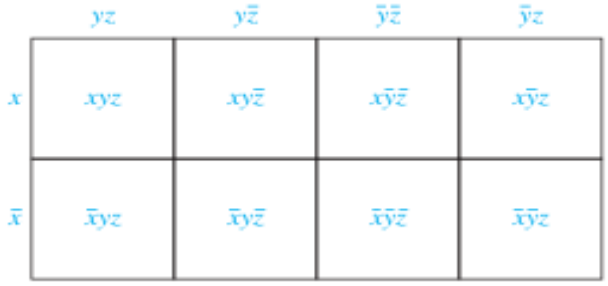
K-maps for the Sum-of-Products Expansions

EXAMPLE 2 Simplify the sum-of-products expansions given in Example-1.

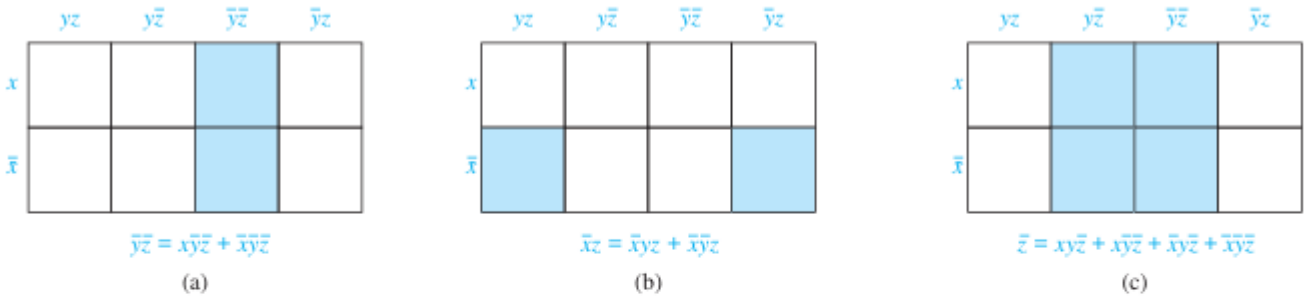
The grouping of minterms is shown in Figure using the K-maps for these expansions. Minimal expansions for these sums-of-products are (a) y, (b) xy' + x'y, and (c) x' + y'.



A K-map in three variables is a rectangle divided into eight cells. The cells represent the eight possible minterms in three variables. Two cells are said to be adjacent if the minterms that they represent differ in exactly one literal. One of the ways to forma K-map in three variables is shown in Figure.



K-maps in Three Variables

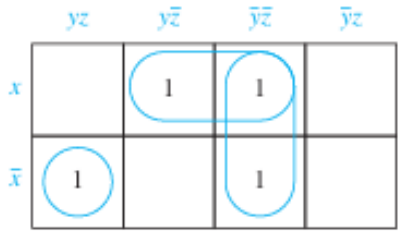


Example-3 illustrate show K-maps in three variables are used.
Use K-maps to minimize these sum-of-products expansions.

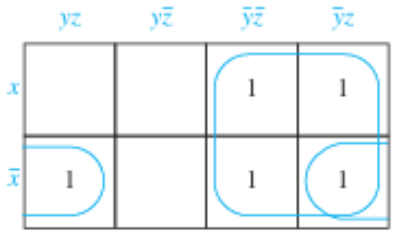
(a) $xy\overline{z} + x\overline{y}\,\overline{z} + \overline{x}yz + \overline{x}\,\overline{y}\,\overline{z}$
(b) $x\overline{y}z + x\overline{y}\,\overline{z} + \overline{x}yz + \overline{x}\,\overline{y}z + \overline{x}\,\overline{y}\,\overline{z}$
(c) $xyz + xy\overline{z} + x\overline{y}z + x\overline{y}\,\overline{z} + \overline{x}yz + \overline{x}\,\overline{y}z + \overline{x}\,\overline{y}\,\overline{z}$
(d) $xy\overline{z} + x\overline{y}\,\overline{z} + \overline{x}\,\overline{y}z + \overline{x}\,\overline{y}\,\overline{z}$

The K-maps for these sum-of-products expansions are shown in Figure, The grouping of blocks shows that minimal expansions into Boolean sums of Boolean products are

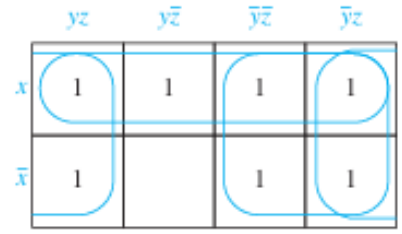
(a) $x\overline{z} + \overline{y}\,\overline{z} + \overline{x}yz$, (b) $\overline{y} + \overline{x}z$, (c) $x + \overline{y} + z$, and (d) $x\overline{z} + \overline{x}\,\overline{y}$.



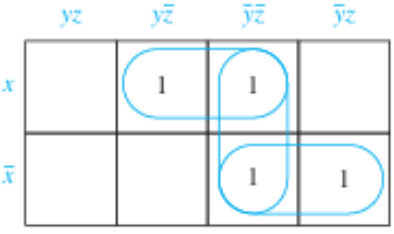
(a)



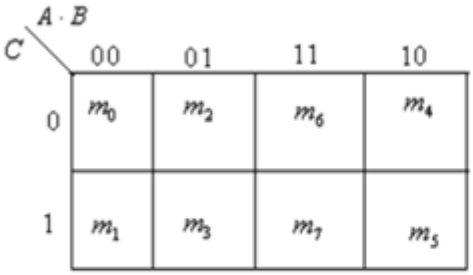
(b)



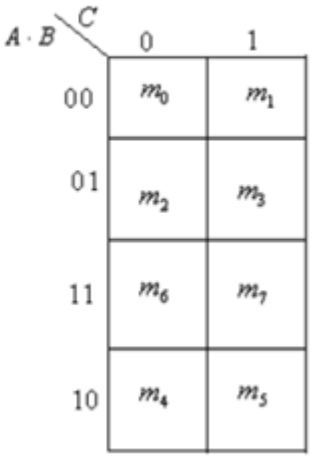
(c)



(d)



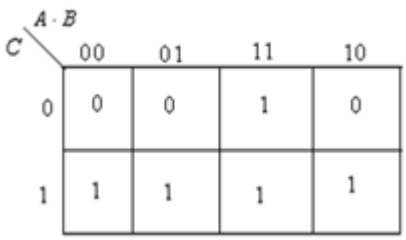
OR



Example: Draw the K – maps for the following Boolean function of three variables.

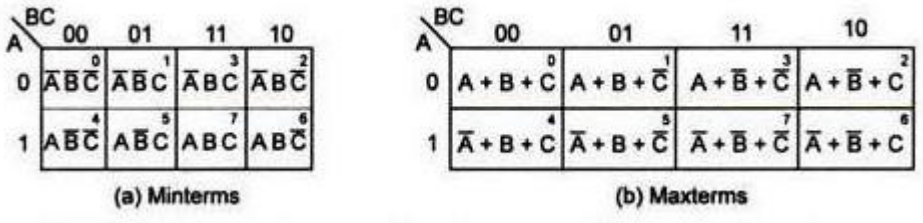
$$F_1(A,B,C) = \sum (m_1, m_3, m_5, m_6, m_7)$$

In the K – map of three variables 1s entry are made for the combinations m1, m3, m5, m6, m7 and in the remaining combinations, 0s are entered.



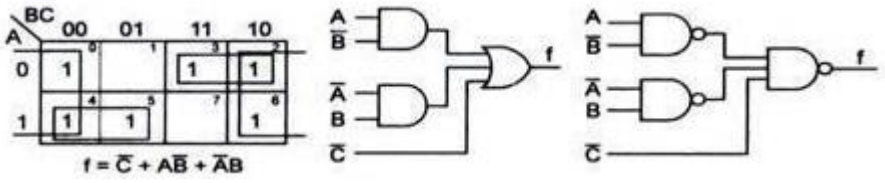
Three-variable K-map

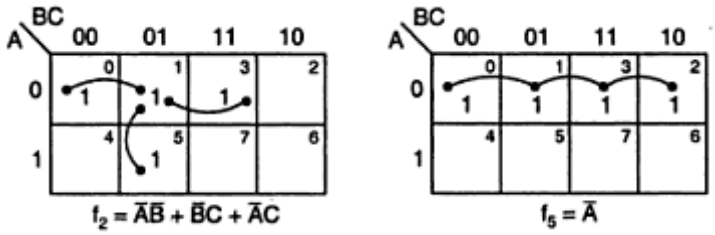
A three-variable k-map has, therefore, 8(=2^3) squares or cells, and each square on the map represents a minterm or maxterm as shown in figure. The small number on the top right corner of each cell indicates the minterm or maxterm designation.



The three-variable k-map

The binary numbers along the top of the map indicate the condition of B and C for each column. The binary number along the left side of the map against each row indicates the condition of A for that row. For example, the binary number 01 on top of the second column in fig indicates that the variable B appears in complemented form and the variable C in non- complemented form in all the minterms in that column. The binary number 0 on the left of the first row indicates that the variable A appears in complemented form in all the minterms in that row, the binary numbers along the top of the k-map are not in normal binary order. They are, in fact, in the Gray code. This is to ensure that two physically adjacent squares are really adjacent, i.e., their minterms or maxterms differ by only one variable.





COMBINATIONAL CIRCUITS

Combinational Logic

Logic circuits for digital systems may be combinational or sequential.

Sequential and combinational logic circuits are the basic building blocks of digital circuits.

The existence of memory elements makes the main difference.

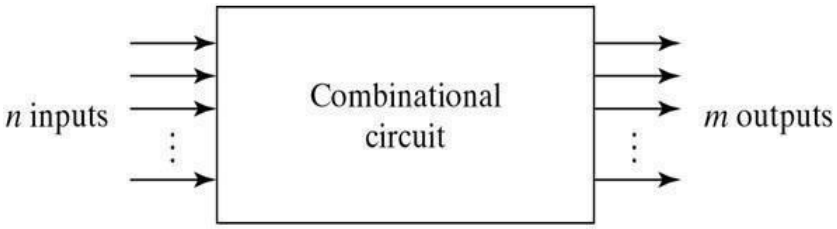
Circuit for **combination doesn't have a memory element**, while a **sequential circuit is composed of memory elements.**

A combinational circuit consists of input variables, logic gates, and output variables.

A combinational circuit could be used to add any number of inputs, or to subtract them, or to perform other mathematical operations.

A **Sequential logic** circuits is a form of binary circuit; its design one or more inputs and one or more outputs,

Examples of such circuits include clocks, flip-flops, bi-stables, counters, memories, and registers.



For n input variables, there are 2^n possible combinations of binary input variables. For each possible input Combination, there is one and only one possible output combination. A combinational circuit can be described by m Boolean functions one for each output variables. Usually the input s comes from flip-flops and outputs go to flip-flops.

Adders:

Digital computers perform variety of information processing tasks, the one is arithmetic operations. And the most basic arithmetic operation is the addition of two binary digits. i.e, 4 basic possible operations are:

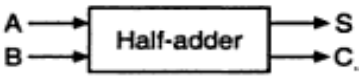
0+0=0, 0+1=1, 1+0=1, 1+1=10

The first three operations produce a sum whose length is one digit, but when addend bits are equal to 1, the binary sum consists of two digits. The higher significant bit of this result is called a carry. A combinational circuit that performs the addition of two bits is called a **half- adder**. One that performs the addition of 3 bits (two significant bits & previous carry) is called a full adder. & **2 half adder can employ as a full-adder.**

The Half Adder: A Half Adder is a combinational circuit with two binary inputs (augends and addend bits and two binary outputs (sum and carry bits.) It adds the two inputs (A and B) and produces the sum (S) and the carry (C) bits. It is an arithmetic operation of addition of two single bit words.

Inputs		Outputs	
A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

(a) Truth table



(b) Block diagram

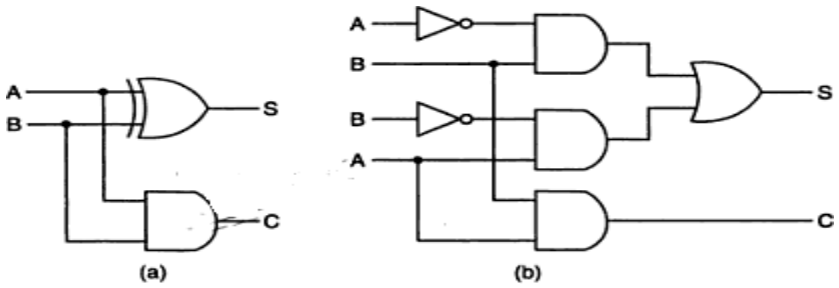
The Sum (S) bit and the carry (C) bit, according to the rules of binary addition, the sum (S) is the X-OR of A and B (It represents the LSB of the sum). Therefore,

$$S=A\overline{B}+\overline{A}B \qquad A \oplus B$$

The carry (C) is the AND of A and B (it is 0 unless both the inputs are 1).Therefore,

$$C=AB$$

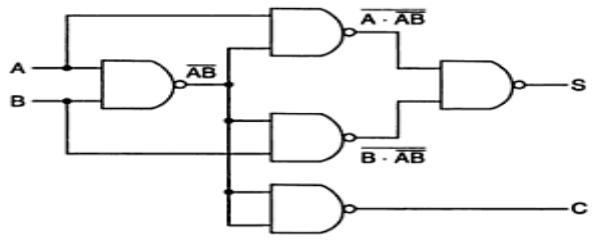
A half-adder can be realized by using one X-OR gate and one AND gate a



Logic diagrams of half-adder

NAND LOGIC:

$$S = A\overline{B} + \overline{A}B = A\overline{B} + A\overline{A} + \overline{A}B + B\overline{B}$$
$$= A(\overline{A} + \overline{B}) + B(\overline{A} + \overline{B})$$
$$= A \cdot \overline{A\overline{B}} + B \cdot \overline{A\overline{B}}$$
$$= \overline{\overline{A \cdot \overline{A\overline{B}}} \cdot \overline{B \cdot \overline{A\overline{B}}}}$$
$$C = AB = \overline{\overline{AB}}$$



Logic diagram of a half-adder using only 2-input NAND gates.

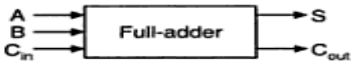
The Full Adder:

A Full-adder is a combinational circuit that adds two bits and a carry and outputs a sum bit and a carry bit. To add two binary numbers, each having two or more bits, the LSBs can be added by using a half-adder. The carry resulted from the addition of the LSBs is carried over to the next significant column and added to the two bits in that column. So, in the second and higher columns, the two data bits of that column and the carry bit generated from the addition in the previous column need to be added.

The full-adder adds the bits A and B and the carry from the previous column called the carry-in C_{in} and outputs the sum bit S and the carry bit called the carry-out C_{out}. The variable S gives the value of the least significant bit of the sum. The variable C_{out} gives the output carry. The eight rows under the input variables designate all possible combinations of 1s and 0s that these variables may have. The 1s and 0s for the output variables are determined from the arithmetic sum of the input bits. When all the bits are 0s, the output is 0. The S output is equal to 1 when only 1 input is equal to 1 or when all the inputs are equal to 1. The C_{out} has a carry of 1 if two or three inputs are equal to 1.

Inputs			Sum	Carry
A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

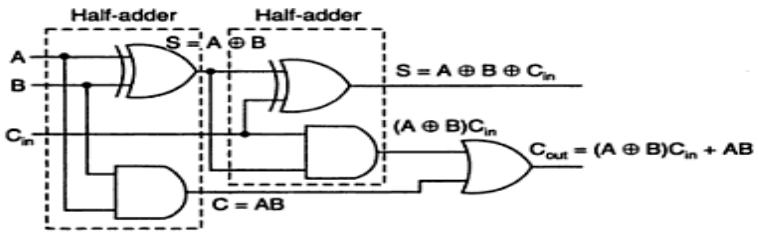
(a) Truth table



(b) Block diagram

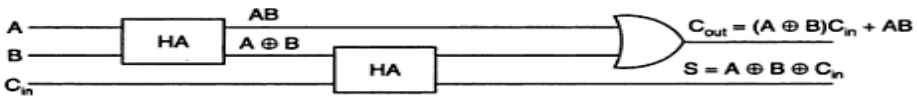
Full-adder.

From the truth table, a circuit that will produce the correct sum and carry bits in response to every possible combination of A,B and C_{in} is described by



Logic diagram of a full-adder using two half-adders.

The block diagram of a full-adder using two half-adders is



Block diagram of a full-adder using two half-adders.

The Full-adder neither can also be realized using universal logic, i.e., either only NAND gates or only NOR gates as

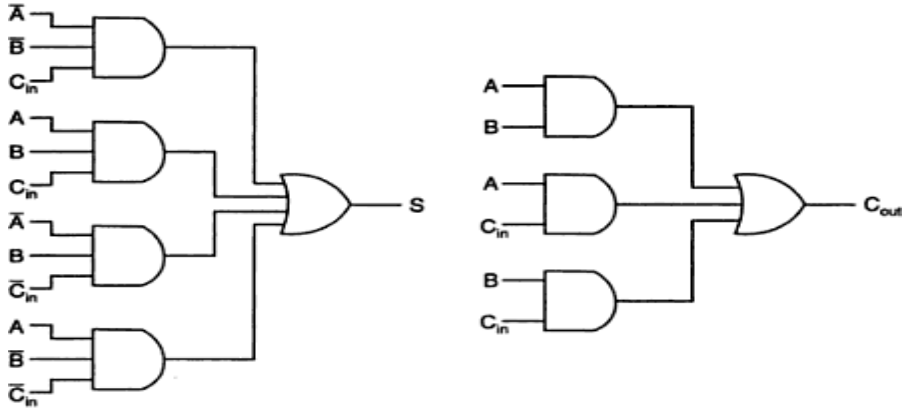
$$A \oplus B = \overline{\overline{A \cdot AB} \cdot \overline{B \cdot AB}}$$

Then

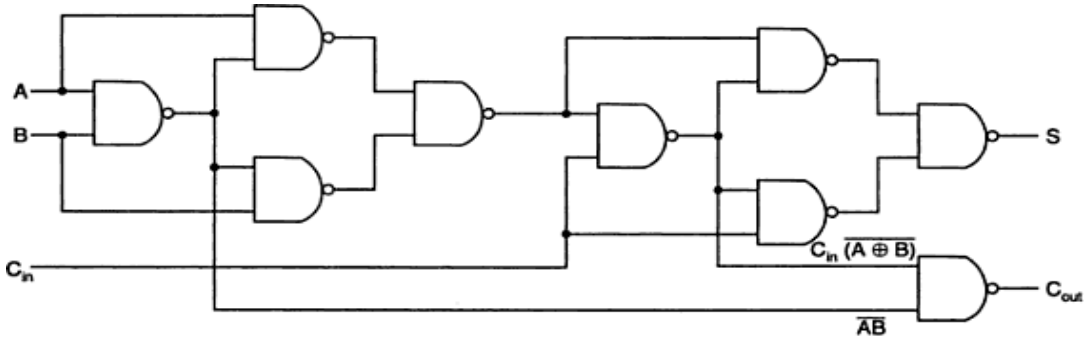
$$S = A \oplus B \oplus C_{in} = \overline{\overline{(A \oplus B) \cdot \overline{(A \oplus B)C_{in}}} \cdot \overline{C_{in} \cdot \overline{(A \oplus B)C_{in}}}}$$

NAND Logic:

$$C_{out} = C_{in}(A \oplus B) + AB = \overline{\overline{C_{in}(A \oplus B)} \cdot \overline{AB}}$$



Sum and carry bits of a full-adder using AOI logic.



Logic diagram of a full-adder using only 2-input NAND gates.

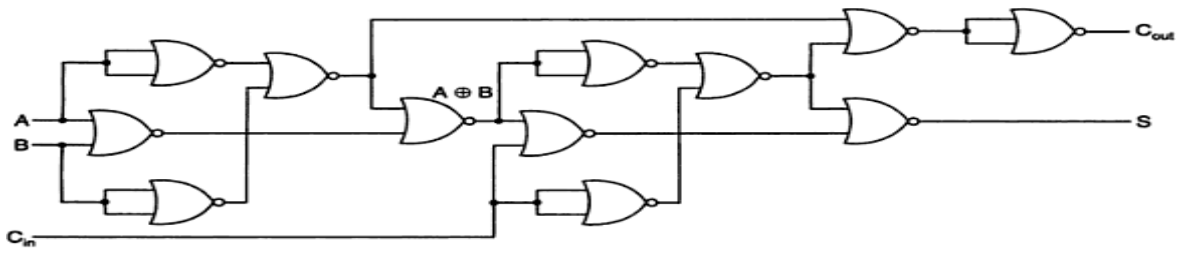
NOR Logic:

$$A \oplus B = \overline{\overline{(A + B)} + \overline{A} + \overline{B}}$$

Then

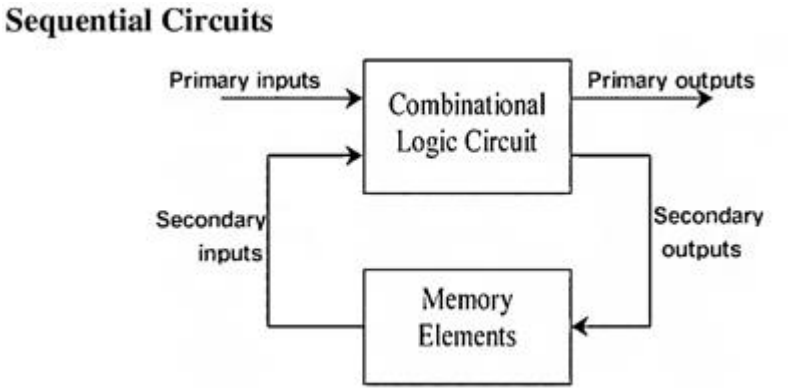
$$S = A \oplus B \oplus C_{in} = \overline{\overline{(A \oplus B) + C_{in}} + \overline{(A \oplus B) + \overline{C}_{in}}}$$

$$C_{out} = AB + C_{in}(A \oplus B) = \overline{\overline{A} + \overline{B} + \overline{C}_{in} + \overline{A \oplus B}}$$



Logic diagram of a full-adder using only 2-input NOR gates.

SEQUENTIAL CIRCUITS



The Basic Latch

Basic latch is a feedback connection of two NOR gates or two NAND gates

It can store one bit of information.

It can be set to 1 using the *S* input and reset to 0 using the *R* input

The Gated Latch

Gated latch is a basic latch that includes input gating and a control signal

The latch retains its existing state when the control input is equal to 0.

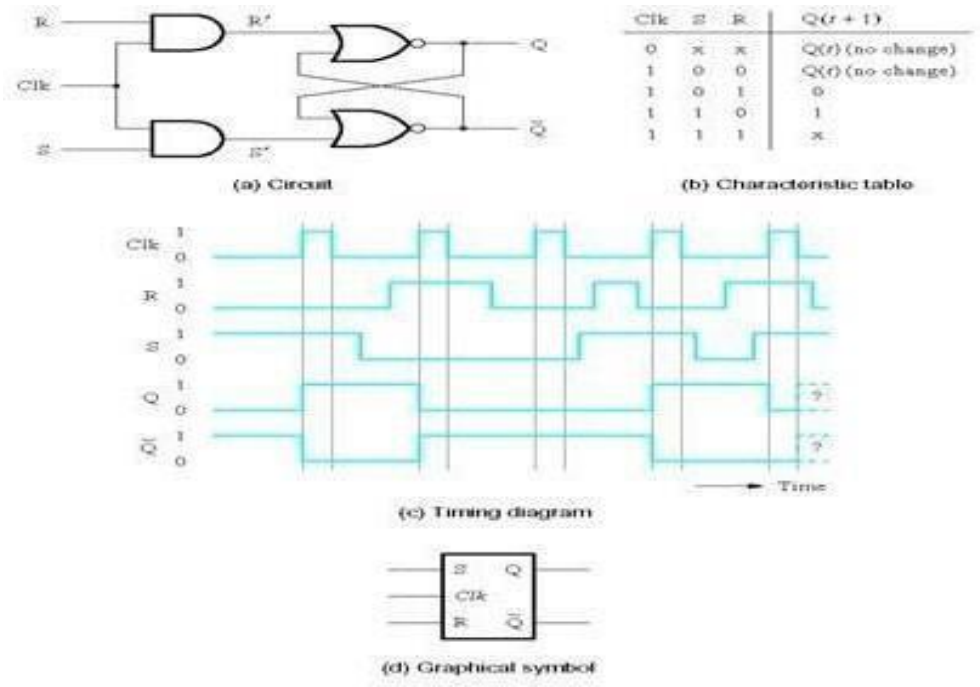
Its state may be changed when the control signal is equal to 1. In our discussion we referred to the control input as the clock.

We consider two types of gated latches:

Gated SR latch uses the *S* and *R* inputs to set the latch to 1 or reset it to 0, respectively.

Gated D latch uses the *D* input to force the latch into a state that has the same logic value as the *D* input.

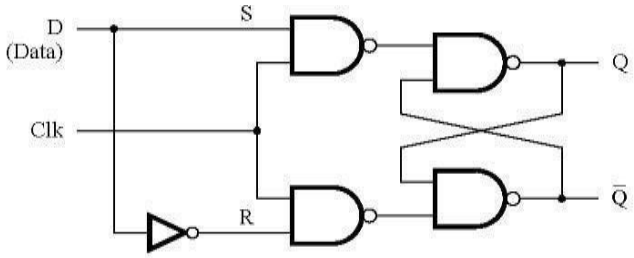
Gated S/R Latch



Gated latches or flip-flops have an extra 'enable' input which controls when they capture the data - in the disabled state, clock changes are ignored.

Latches are either open or closed doors.

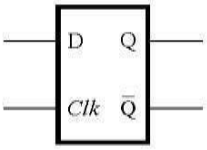
Gated D Latch: uses the *D* input to force the latch into a state that has the same logic value as the *D* input.



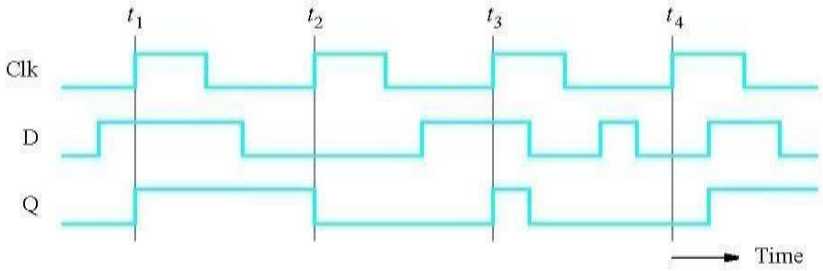
(a) Circuit

Clk	D	Q(t + 1)
0	x	Q(t)
1	0	0
1	1	1

(b) Characteristic table



(c) Graphical symbol



(d) Timing diagram

Flip-Flops

A **flip-flop** an application of logic gates, a storage element based on the gated latch principle

It can have its output state changed only on the edge of the controlling clock signal.

The most commonly used application of flip flops is in the implementation of a feedback circuit. As a memory relies on the feedback concept, flip flops can be used to design it.

Flip-flops are widely used in registers, counters, and memory devices.

Edge-triggered flip-flop is affected only by the input values present when the active edge of the clock occurs.

Master-slave flip-flop is built with two gated latches.

The master stage is active during half of the clock cycle, and the slave stage is active during the other half.
The output value of the flip-flop changes on the edge of the clock that activates the transfer into the slave stage.

Four types of flip flops that are used in electronic circuits.

1. **The basic Flip Flop or S-R Flip Flop**
2. **Delay Flip Flop [D Flip Flop]**
3. **J-K Flip Flop**
4. **T Flip Flop**

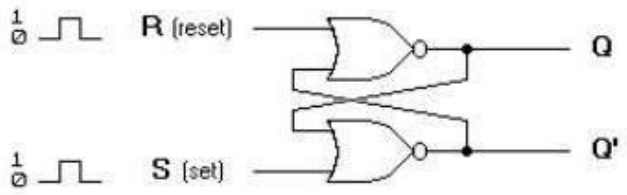
S-R Flip Flop

The SET-RESET flip flop is designed with the help of two NOR gates and also two NAND gates. These flip flops are also called S-R Latch.

Basic bistable circuit with Set (S) and Reset (R) inputs.

S-R Flip Flop using NOR Gate

The design of such a flip flop includes two inputs, called the SET [S] and RESET [R]. There are also two outputs, Q and Q'. The diagram and truth table is shown below.



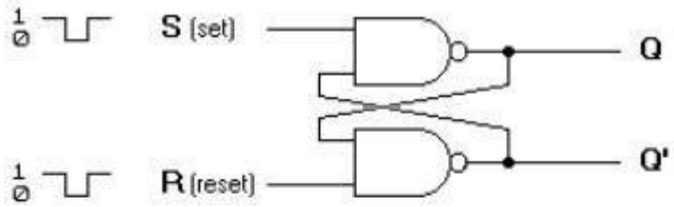
(a) Logic diagram

S	R	Q	Q'	
1	0	1	0	
0	0	1	0	(after S=1, R=0)
0	1	0	1	
0	0	0	1	(after S=0, R=1)
1	1	0	0	

(b) Truth table

Basic flip-flop circuit with NOR gates

S-R Flip Flop using NAND Gate



(a) Logic diagram

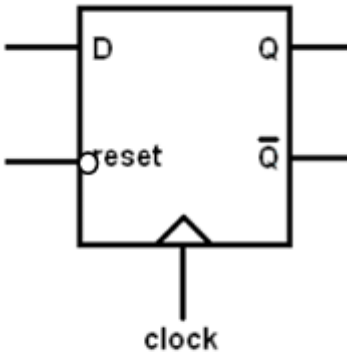
S	R	Q	Q'	
1	0	0	1	
1	1	0	1	(after S=1, R=0)
0	1	1	0	
1	1	1	0	(after S=0, R=1)
0	0	1	1	

(b) Truth table

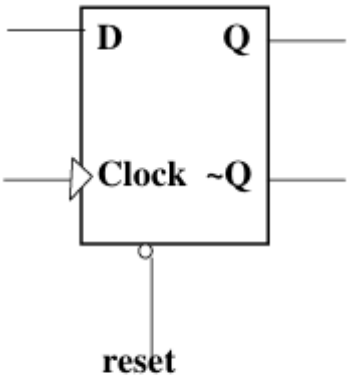
Basic flip-flop circuit with NAND gates

Synchronous and asynchronous circuits

SYNCHRONOUS CIRCUITS	ASYNCHRONOUS CIRCUITS
1) These are the sequential circuits which are governed by a global clock signal.	1) These are circuits which are governed by a input signals regardless of clock signal.
2) The state of all elements of a synchronous circuit changes only by an application of a distributed clock signal.	2) Asynchronous circuits change state through the inputs received by them.

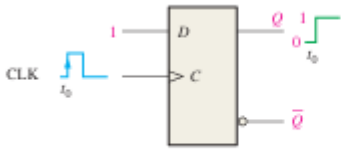


Synchronous D flip flop

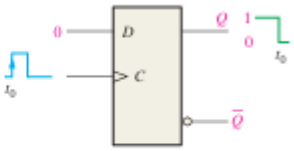


Asynchronous D flip flop

The D Flip-Flop



(a) $D = 1$ flip-flop SETS on positive clock edge. (If already SET, it remains SET.)



(b) $D = 0$ flip-flop RESETS on positive clock edge. (If already RESET, it remains RESET.)

Truth table for a positive edge-triggered D flip-flop.

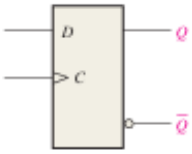
Inputs		Outputs		
D	CLK	Q	$\overline{Q}$	Comments
0	↑	0	1	RESET
1	↑	1	0	SET

↑ = clock transition LOW to HIGH

Operation of a positive edge-triggered D flip-flop.

EXAMPLE

Determine the Q and Q' output waveforms of the flip-flop in Figure, for the D and CLK inputs in Figure (a). Assume that the positive edge-triggered flip-flop is initially RESET.



Figure

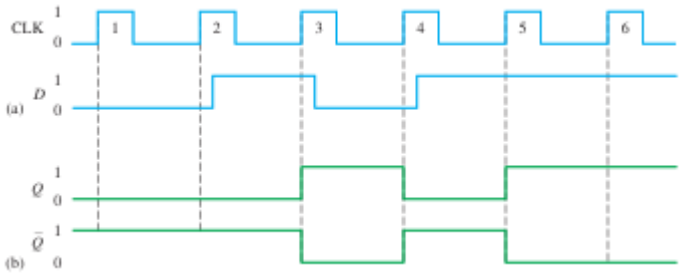
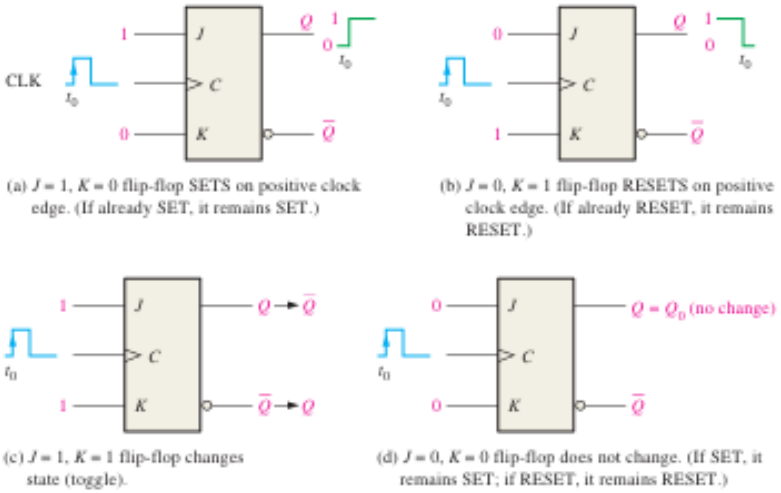


Figure (a)

Solution

At clock pulse 1, D is LOW, so Q remains LOW (RESET).
At clock pulse 2, D is LOW, so Q remains LOW (RESET).
At clock pulse 3, D is HIGH, so Q goes HIGH (SET).
At clock pulse 4, D is LOW, so Q goes LOW (RESET).
At clock pulse 5, D is HIGH, so Q goes HIGH (SET).
At clock pulse 6, D is HIGH, so Q remains HIGH (SET).
Once Q is determined, Q' is easily found since it is simply the complement of Q. The resulting waveforms for Q and Q' are shown in Figure (a) and (b) for the input waveforms.

The J-K Flip-Flop



(a) $J = 1$, $K = 0$ flip-flop SETS on positive clock edge. (If already SET, it remains SET.)

(b) $J = 0$, $K = 1$ flip-flop RESETS on positive clock edge. (If already RESET, it remains RESET.)

(c) $J = 1$, $K = 1$ flip-flop changes state (toggle).

(d) $J = 0$, $K = 0$ flip-flop does not change. (If SET, it remains SET; if RESET, it remains RESET.)

Operation of a positive edge-triggered J-K flip-flop

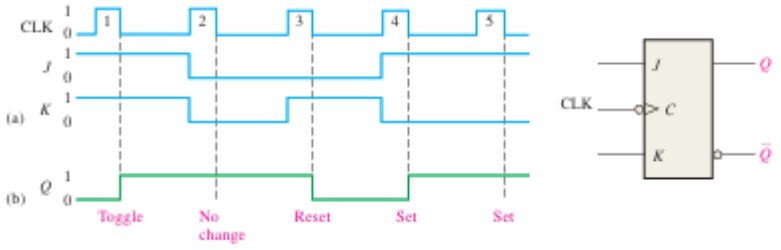
Truth table for a positive edge-triggered J-K flip-flop.

Inputs			Outputs		
J	K	CLK	Q	$\overline{Q}$	Comments
0	0	↑	Q_0	$\overline{Q}_0$	No change
0	1	↑	0	1	RESET
1	0	↑	1	0	SET
1	1	↑	$\overline{Q}_0$	Q_0	Toggle

↑ = clock transition LOW to HIGH
Q_0 = output level prior to clock transition

EXAMPLE

The waveforms in Figure are applied to the J, K, and clock inputs as indicated. Determine the Q output, assuming that the flip-flop is initially RESET.



Since this is a negative edge-triggered flip-flop, as indicated by the "**bubble**" at the clock input, the Q output will change only on the negative-going edge of the clock pulse.

1. At the first clock pulse 1, both J and K are HIGH; and because this is a toggle condition, Q goes HIGH.
2. At clock pulse 2, a no-change condition exists on the inputs, keeping Q at a HIGH level.
3. When clock pulse 3 occurs, J is LOW and K is HIGH, resulting in a RESET condition; Q goes LOW.
4. At clock pulse 4, J is HIGH and K is LOW, resulting in a SET condition; Q goes HIGH.
5. A SET condition still exists on J and K when clock pulse 5 occurs, so Q will remain HIGH.
   The resulting Q waveform is indicated in Figure (b).

Shift Registers

Shift registers consist of arrangements of flip-flops and are important in applications involving the **storage and transfer of data in a digital system**. A register can consist of one or more flip-flops used to store and shift data.

A register has no specified sequence of states, except in certain very specialized applications. A register, in general, is used solely for storing and shifting data (1s and 0s) entered into it from an external source and typically possesses no characteristic internal sequence of states.

A register is a digital circuit with two basic functions: data storage and data movement. The storage capability of a register makes it an important type of memory device. Figure illustrates the concept of storing a 1 or a 0 in a D flip-flop. A 1 is applied to the data input as shown, and a clock pulse is applied that stores the 1 by setting the flip-flop. When the 1 on the input is removed, the flip-flop remains in the SET state, thereby storing the 1. A similar procedure applies to the storage of a 0 by resetting the flip-flop, as also illustrated in Figure.

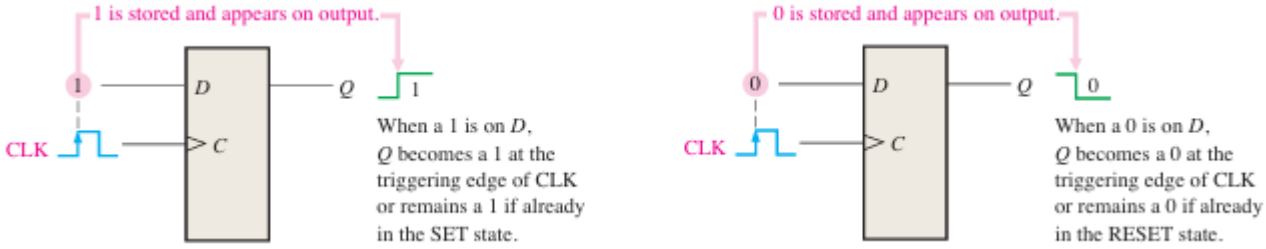
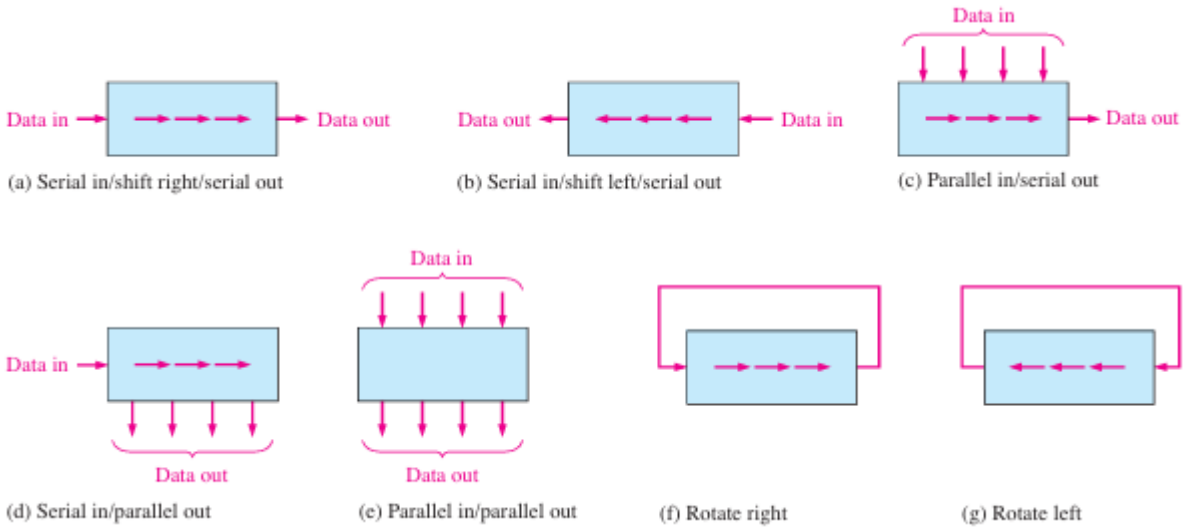


FIGURE-flip-flop as a storage element.

The storage capacity of a register is the total number of bits (1s and 0s) of digital data it can retain. Each stage (flip-flop) in a shift register represents one bit of storage capacity; therefore, the number of stages in a register determines its storage capacity. The shift capability of a register permits the movement of data from stage to stage within the register or into or out of the register upon application of clock pulses. Figure.



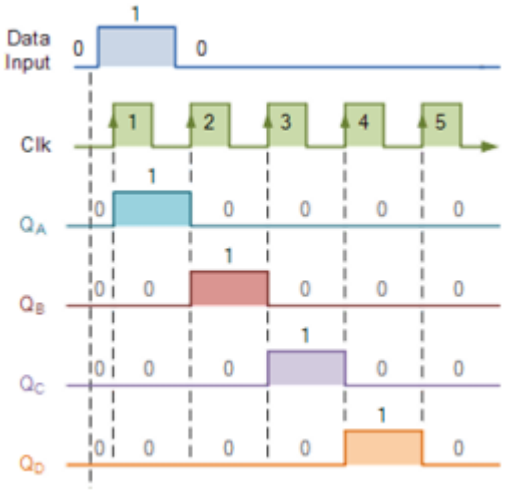
Basic data movement in shift registers. Four bits are used for illustration. The bits move in direction of the arrows

Illustrates the types of data movement in shift registers. The block represents any arbitrary 4-bit register, and the arrows indicate the direction of data movement.

This sequential device loads the data present on its inputs and then moves or “shifts” it to its output once every clock cycle, hence the name Shift Register.

Basic Data Movement through a Shift Register

Clock Pulse No	QA	QB	QC	QD
0	0	0	0	0
1	1	0	0	0
2	0	1	0	0
3	0	0	1	0
4	0	0	0	1
5	0	0	0	0



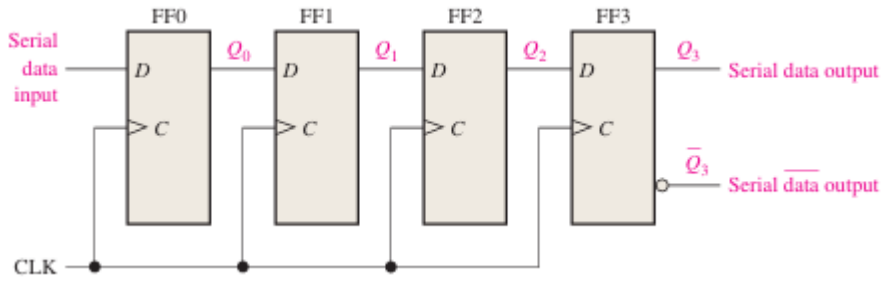
Types of Shift Register Data I/Os

Four types of shift registers based on data input and output (inputs/outputs)

- Serial in/serial out
- Serial in/parallel out
- Parallel in/serial out
- Parallel in/parallel out

Serial In/Serial out Shift Registers

The serial in/serial out shift register accepts data serially—that is, one bit at a time on a single line. It produces the stored information on its output also in serial form. Let’s first look at the serial entry of data into a typical shift register. Figure shows a 4-bit device implemented with D flip-flops. With four stages, this register can store up to four bits of data.



For serial data, one bit at a time is transferred

Table shows the entry of the four bits 1010 into the register in Figure, beginning with the least significant bit. The register is initially clear. The 0 is put onto the data input line, making D = 0 for FF0. When the first clock pulse is applied, FF0 is reset, thus storing the 0.

CLK	FF0 (Q_0)	FF1 (Q_1)	FF2 (Q_2)	FF3 (Q_3)
Initial	0	0	0	0
1	**0**	0	0	0
2	**1**	**0**	0	0
3	**0**	**1**	**0**	0
4	**1**	**0**	**1**	**0**

Shifting a 4-bit code into the shift register

Next the second bit, which is a 1, is applied to the data input, making D = 1 for FF0 and D = 0 for FF1 because the D input of FF1 is connected to the Q**0** output. When the second clock pulse occurs, the 1 on the data input is shifted into FF0, causing FF0 to set; and the 0 that was in FF0 is shifted into FF1.

The third bit, a 0, is now put onto the data-input line, and a clock pulse is applied. The 0 is entered into FF0, the 1 stored in FF0 is shifted into FF1, and the 0 stored in FF1 is shifted into FF2.

The last bit, a 1, is now applied to the data input, and a clock pulse is applied. This time the 1 is entered into FF0, the 0 stored in FF0 is shifted into FF1, the 1 stored in FF1 is shifted into FF2, and the 0 stored in FF2 is shifted into FF3.
This completes the serial entry of the four bits into the shift register, where they can be stored for any length of time as long as the flip-flops have dc power.

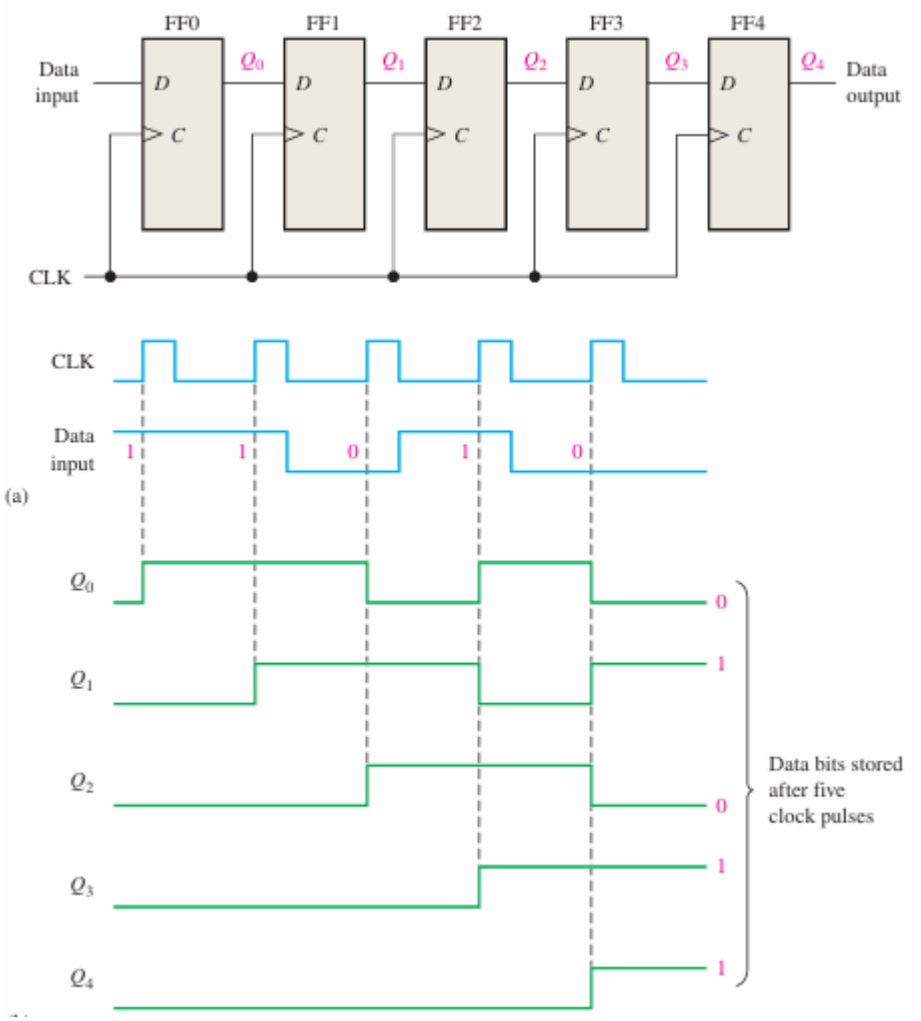
If you want to get the data out of the register, the bits must be shifted out serially to the Q3 output, as Table illustrates.
After CLK4 in the data-entry operation just described, the LSB 0, appears on the Q3 output. When clock pulse CLK5 is applied, the second bit appears on the Q3 output. Clock pulse CLK6 shifts the third bit to the output, and CLK7 shifts the fourth bit to the output. While the original four bits are being shifted out, more bits can be shifted in. All zeros are shown being shifted in, after CLK8.

CLK	FF0 (Q_0)	FF1 (Q_1)	FF2 (Q_2)	FF3 (Q_3)
Initial	**1**	**0**	**1**	**0**
5	0	**1**	**0**	**1**
6	0	0	**1**	**0**
7	0	0	0	**1**
8	0	0	0	0

Shifting a 4-bit code out of the shift register

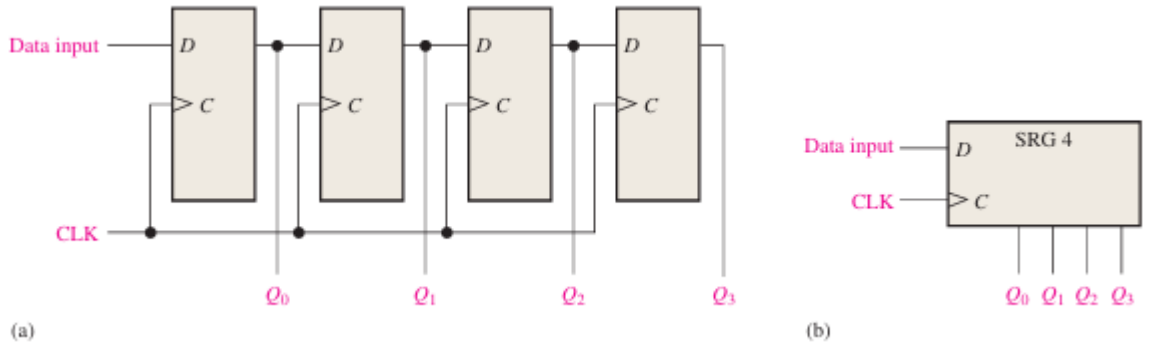
EXAMPLE:
Show the states of the 5-bit register in Figure for the specified data input and clock waveforms. Assume that the register is initially cleared (all 0s).

Solution
The first data bit (1) is entered into the register on the first clock pulse and then shifted from left to right as the remaining bits are entered and shifted. The register contains Q4 Q3 Q2 Q1 Q0 = 11010 after five clock pulses. See Figure.

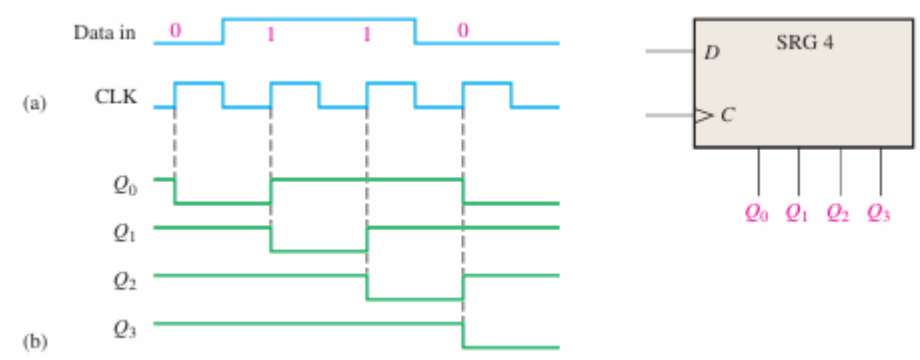


Serial In/Parallel out Shift Registers

Data bits are entered serially (least-significant bit first) into a serial in/parallel out shift register in the same manner as in serial in/serial out registers. The difference is the way in which the data bits are taken out of the register; in the parallel output register, the output of each stage is available. Once the data are stored, each bit appears on its respective output line, and all bits are available simultaneously, rather than on a bit-by-bit basis as with the serial output. Figure shows a 4-bit serial in/parallel out shift register and its logic block symbol.



E.g.: Show the states of the 4-bit register (SRG 4) for data input and clock waveforms in Figure. The register initially contains all 1s.

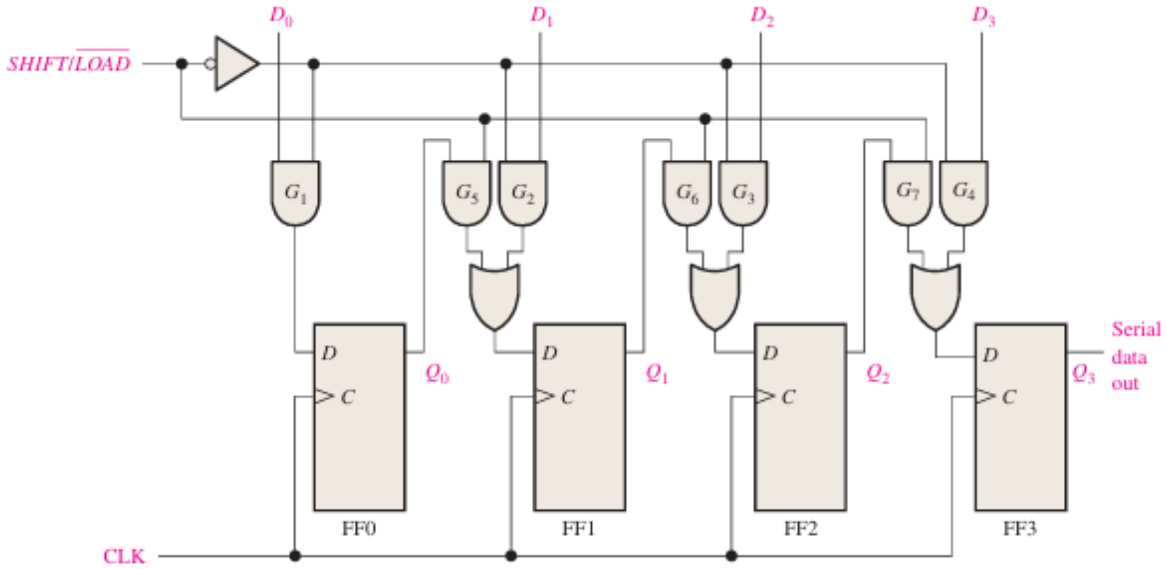


Solution

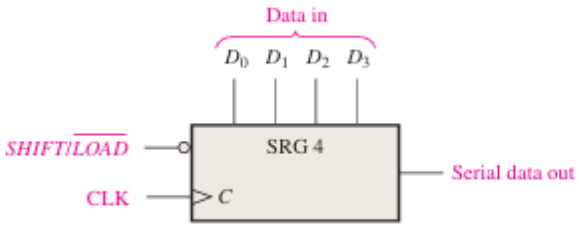
The register contains 0110 after four clock pulses. See Figure (b)

Parallel In/Serial out Shift Registers For parallel data, multiple bits are transferred at one time

For a register with parallel data inputs, the bits are entered simultaneously into their respective stages on parallel lines rather than on a bit-by-bit basis on one line as with serial data inputs. The serial output is the same as in serial in/serial out shift registers, once the data are completely stored in the register. Figure illustrates a 4-bit parallel in/serial out shift register and a typical logic symbol. There are four data-input lines, D0, D1, D2, and D3, and a SHIFT/LOAD input, which allows four bits of data to load in parallel into the register. When SHIFT/LOAD is LOW, gates G1 through G4 are enabled, allowing each data bit to be applied to the D input of its respective flip-flop. When a clock pulse is applied, the flip-flops with D=1 will set and those with D=0 will reset, thereby storing all four bits simultaneously. When SHIFT/LOAD is HIGH, gates G1 through G4 are disabled and gates G5 through G7 are enabled, allowing the data bits to shift right from one stage to the next. The OR gates allow either the normal shifting operation or the parallel data-entry operation, depending on which AND gates are enabled by the level on the SHIFT/LOAD input. Notice that FF0 has a single AND to disable the parallel input, D0. It does not require an AND/OR arrangement because there is no serial data in.



(a) Logic diagram



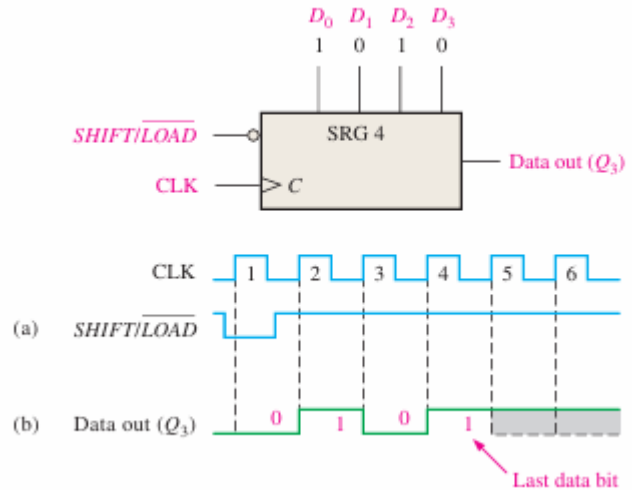
(b) Logic symbol

EXAMPLE
Show the data-output waveform for a 4-bit register with the parallel input data and the clock and SHIFT/LOAD waveforms given in Figure (a). Refer to Figure for the logic diagram.

Solution

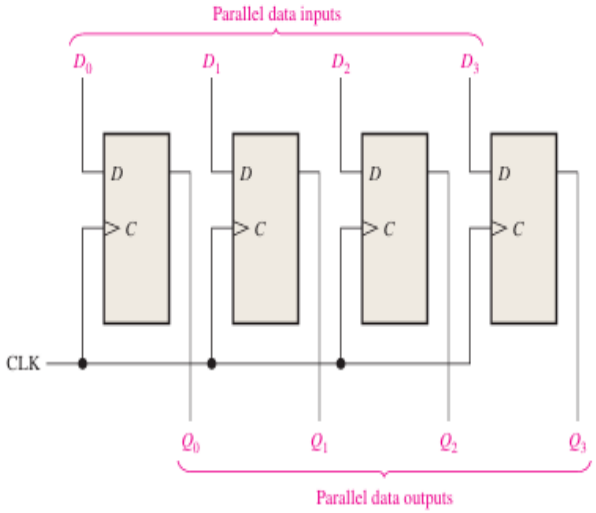
On clock pulse 1, the parallel data (D0 D1 D2 D3 = 1010) are loaded into the register, making Q3 a 0. On clock pulse 2 the 1 from Q2 is shifted onto Q3; on clock pulse 3 the 0 is shifted onto Q3; on clock pulse 4 the last data bit (1) is shifted onto Q3; and on clock pulse 5, all data bits have been shifted out, and only 1s remain in the register (assuming the D0 input remains a 1)

Clock Pulse	Load Enable	Parallel Input (P3-P0)	Q3 (MSB)	Q2	Q1	Q0 (LSB)	Serial Out (Q0)
0 (Initial)	1 (Load)	**1 0 1 0**	1	0	1	0	- (Not Shifted)
1	0 (Shift)	-	0	1	0	1	**0**
2	0 (Shift)	-	1	0	1	0	**1**
3	0 (Shift)	-	0	1	0	-	**0**
4	0 (Shift)	-	1	0	-	-	**1**



Parallel In/Parallel Out Shift Registers

The parallel in/parallel out register employs both methods. Immediately following the simultaneous entry of all data bits, the bits appear on the parallel outputs. Figure shows a parallel in/parallel out shift register.



D flip-flops are used in shift registers because they are the simplest and most suitable type of flip-flop with single input for storing and shifting data.

The shift Registers can also be designed with RS flip-flops by inverting R input with respect to S input.

JK and T flip flops are good for optimizing the hard ware design that is to minimize the gate count. There is no such need in a shift register.

Finite State Machines:

A state machine is a sequential circuit having a limited (finite) number of states occurring in a prescribed order. A counter is an example of a state machine; the number of states is called the modulus.

Two basic types of state machines are the Moore and the Mealy.

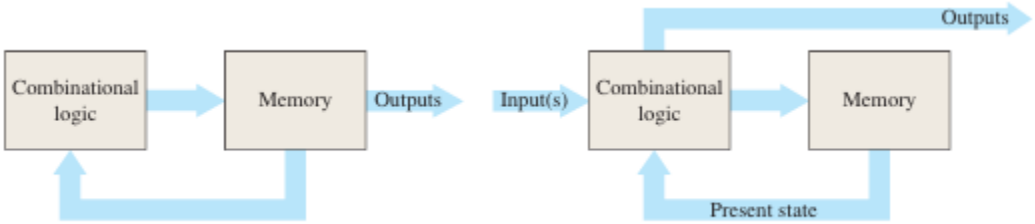
The Moore state machine is one where the outputs depend only on the internal present state.

The Mealy state machine is one where the outputs depend on both the internal present state and on the inputs.

Both types have a timing input (clock) that is not considered a controlling input. A design approach to counters is presented.

General Models of Finite State Machines:

A Moore state machine consists of combinational logic that determines the sequence and memory (Flip-flops), as shown in Figure (a). A Mealy state machine is shown in part (b).



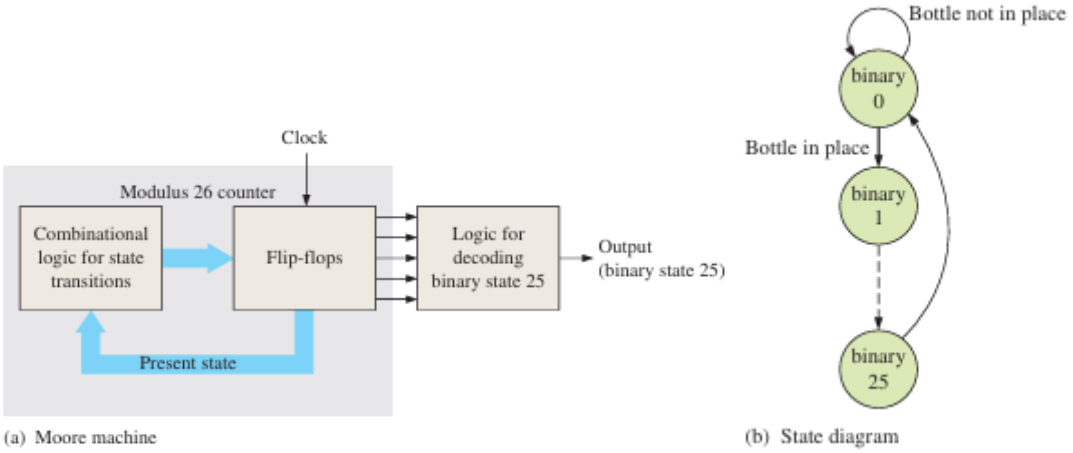
(a) Moore machine (b) Mealy machine

In the Moore machine, the combinational logic is a gate array with outputs that determine the next state of the flip-flops in the memory. There may or may not be inputs to the combinational logic. There may also be output combinational logic, such as a decoder. If there is an input(s), it does not affect the outputs because they always correspond to and are dependent only on the present state of the memory.

For the Mealy machine, the present state affects the outputs, just as in the Moore machine; but in addition, the inputs also affect the outputs. The outputs come directly from the combinational logic and not the memory.

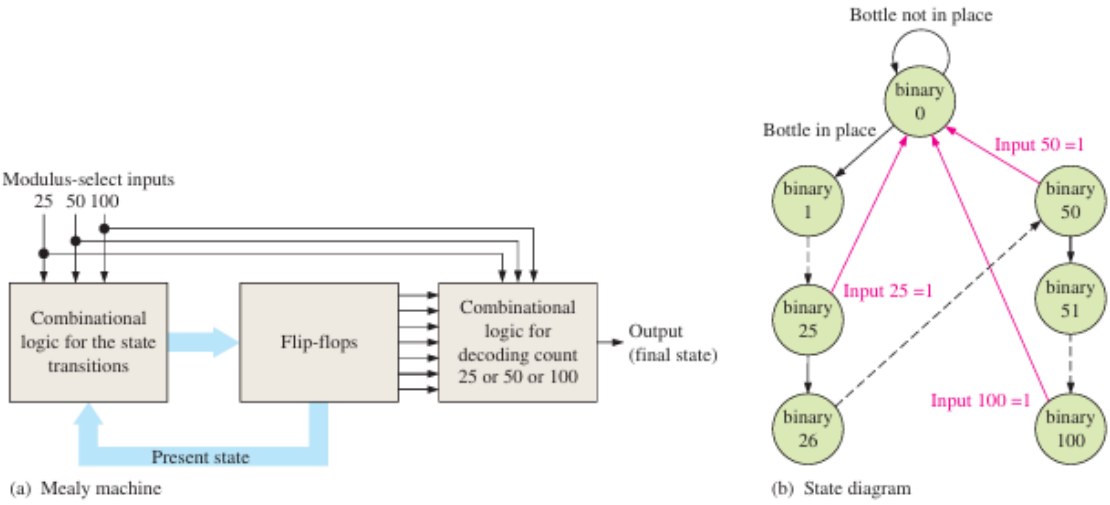
Example of a Moore Machine:

Figure (a) shows a Moore machine (modulus-26 binary counter with states 0 through 25) that is used to control the number of tablets (25) that go into each bottle in an assembly line. When the binary number in the memory (flip-flops) reaches binary twenty-five (11001), the counter recycles to 0 and the tablet flow and clock are cut off until the next bottle is in place. The combinational logic for the state transitions sets the modulus of the counter so that it sequences from binary state 0 to binary state 25, where 0 is the reset or rest state and the output combinational logic decodes binary state 25. There is no input in this case, other than the clock, so the next state is determined only by the present state, which makes this a Moore machine. One tablet is bottled for each clock pulse. Once a bottle is in place, the first tablet is inserted at binary state 1, the second at binary state 2, and the twenty-fifth tablet when the binary state is 25. Count 25 is decoded and used to stop the flow of tablets and the clock. The counter stays in the 0 state until the next bottle is in position (indicated by a 1). Then the clock resumes, the count goes to 1, and the cycle repeats, as illustrated by the state diagram in Figure (b)



Example of a Mealy Machine:

Let's assume that the tablet-bottling system uses three different sizes of bottles:
a 25-tablet bottle, a 50-tablet bottle, and a 100-tablet bottle.
This operation requires a state machine with three different terminal counts: 25, 50, and 100.
One approach is illustrated in Figure (a). The combinational logic sets the modulus of the counter depending on the modulus-select inputs. The output of the counter depends on both the present state and the modulus-select inputs, making this a Mealy machine. The state diagram is shown in part (b).



A variable-modulus binary counter as an example of a Mealy state machine.
The red arrows in the state diagram represent the recycle paths that depend on the input number.
The black dashed lines mean the interim states are not shown for simplicity.

Introduction to Counters

A counter may count up or count down depending on the input control.

Asynchronous Counters (or Ripple counters):
The clock signal (CLK) is only used to clock the first FF.
Each FF (except the first FF) is clocked by the preceding FF.

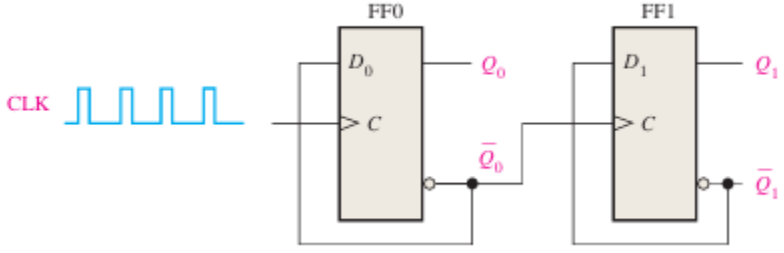
Synchronous Counters:
The clock signal (CLK) is applied to all FF, which means that all FF shares the same clock signal

Asynchronous Counters:
The term asynchronous refers to events that do not have a fixed time relationship with each other and, generally, do not occur at the same time. An asynchronous counter is one in which the flip-flops (FF) within the counter do not change states at exactly the same time because they do not have a common clock pulse.

The most typical uses of counters:
To count the number of times that a certain event takes place;
The occurrence of event to be counted is represented by the input signal to the counter.
To control a fixed sequence of actions in a digital system.
To generate timing signals.
To generate clocks of different frequencies.

ADVANTAGES: Asynchronous counters can be easily designed by T flip flop.
These are also called as Ripple counters, and are used in low speed circuits.

A 2-Bit Asynchronous Binary Counter:
The clock input of an asynchronous counter is always connected only to the LSB flip-flop.
Figure shows a 2-bit counter connected for asynchronous operation. Notice that the clock (CLK) is applied to the clock input (C) of only the first flip-flop, FF0, which is always the least significant bit (LSB). The second flip-flop, FF1, is triggered by the Q0' output of FF0. FF0 changes state at the positive-going edge of each clock pulse, but FF1 changes only when triggered by a positive-going transition of the Q0' output of FF0. Because of the inherent propagation delay time through a flip-flop, a transition of the input clock pulse (CLK) and a transition of the Q0' output of FF0 can never occur at exactly the same time. Therefore, the two flip-flops are never simultaneously triggered, so the counter operation is asynchronous.

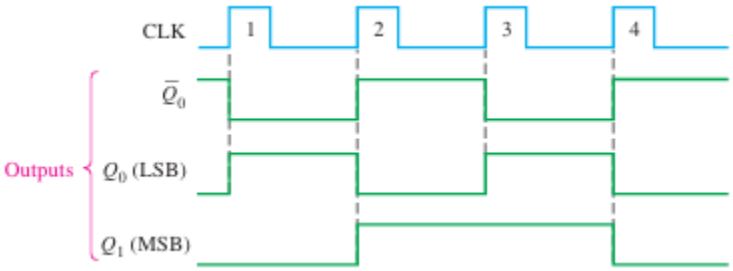


It has 2 unique output states (0 and 1). 2 bit asynchronous counter. When two FFs are connected in series and output of one FF is act as clock for 2nd FF. So the state of 2nd FF will change only when output and 1st FF is logic 1 and falling edge occur.

The Timing Diagram:

Let's examine the basic operation of the asynchronous counter of Figure 9–4 by applying four clock pulses to FF0 and observing the Q output of each flip-flop. Figure 9–5 illustrates the changes in the state of the flip-flop outputs in response to the clock pulses. Both flip-flops are connected for toggle operation (D = Q) and are assumed to be initially RESET (Q LOW).

The positive-going edge of CLK1 (clock pulse 1) causes the Q0 output of FF0 to go HIGH, as shown in Figure 9–5. At the same time the Q0 output goes LOW, but it has no effect on FF1 because a positive-going transition must occur to trigger the flip-flop. After the leading edge of CLK1, Q0 = 1 and Q1 = 0. The positive-going edge of CLK2 causes Q0 to go LOW. Output Q0 goes HIGH and triggers FF1, causing Q1 to go HIGH. After the leading edge of CLK2, Q0 = 0 and Q1 = 1. The positive-going edge of CLK3 causes Q0 to go HIGH again. Output Q0 goes LOW and has no effect on FF1. Thus, after the leading edge of CLK3, Q0 = 1 and Q1 = 1. The positive-going edge of CLK4 causes Q0 to go LOW, while Q0 goes HIGH and triggers FF1, causing Q1 to go LOW. After the leading edge of CLK4, Q0=0 and Q1=0. The counter has now recycled to its original state (both flip-flops are RESET).



Binary state sequence for the counter in Figure 9–4.

Clock Pulse	Q_1	Q_0
Initially	0	0
1	0	1
2	1	0
3	1	1
4 (recycles)	0	0

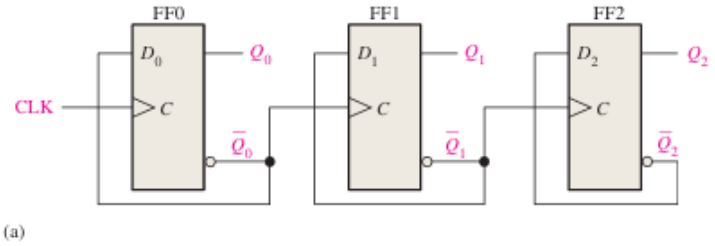
Since it goes through a binary sequence, the counter in Figure 9–4 is a binary counter. It actually counts the number of clock pulses up to three, and on the fourth pulse it recycles to its original state (Q0=0, Q1=0). The term recycle is commonly applied to counter operation; it refers to the transition of the counter from its final state back to its original state.

A 3-Bit Asynchronous Binary Counter:

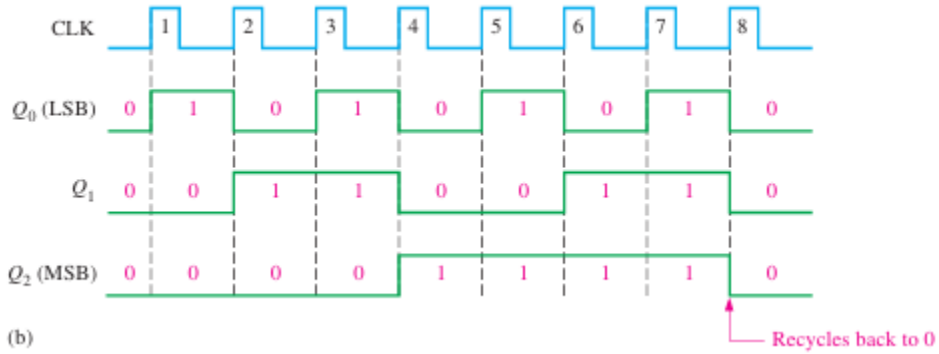
The state sequence for a 3-bit binary counter is listed in Table, and a 3-bit asynchronous binary counter is shown in Figure a. The basic operation is the same as that of the 2-bit counter except that the 3-bit counter has eight states, due to its three flip-flops. A timing diagram is shown in Figure b for eight clock pulses.

State sequence for a 3-bit binary counter.

Clock Pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0



(a)



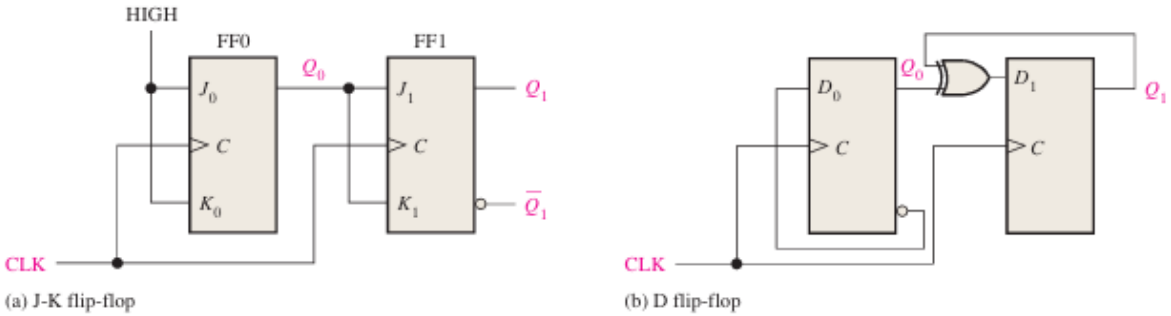
(b)

Synchronous Counters:

The term synchronous refers to events that have a fixed time relationship with each other. A synchronous counter is one in which all the flip-flops in the counter are clocked at the same time by a common clock pulse. J-K flip-flops are used to most synchronous counters. D flip-flops can also be used but generally require more logic because of having no direct toggle or no-change states.

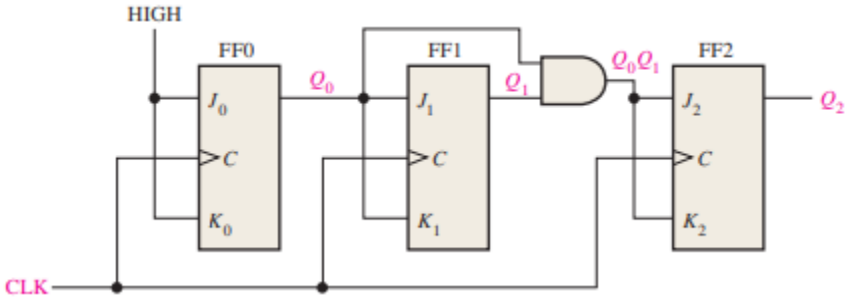
A 2-Bit Synchronous Binary Counter

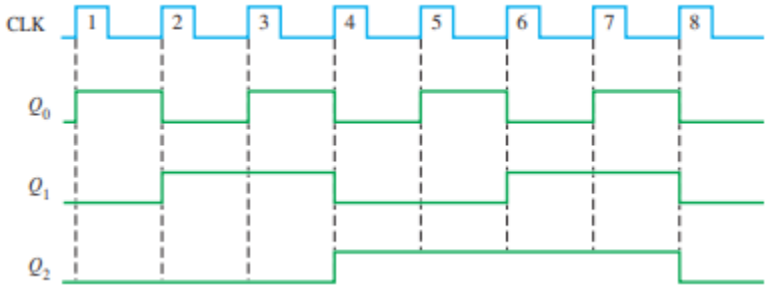
Figure shows a 2-bit synchronous binary counter. Notice that an arrangement different from that for the asynchronous counter must be used for the J1 and K1 inputs of FF1 in order to achieve a binary sequence. A D flip-flop implementation is shown in part (b).



The clock input goes to each flip-flop in a synchronous counter

A 3-Bit Synchronous Binary Counter:
A 3-bit synchronous binary counter is shown in Figure, and its timing diagram is shown. You can understand this counter operation by examining its sequence of states as shown in Table.





State sequence for a 3-bit binary counter.

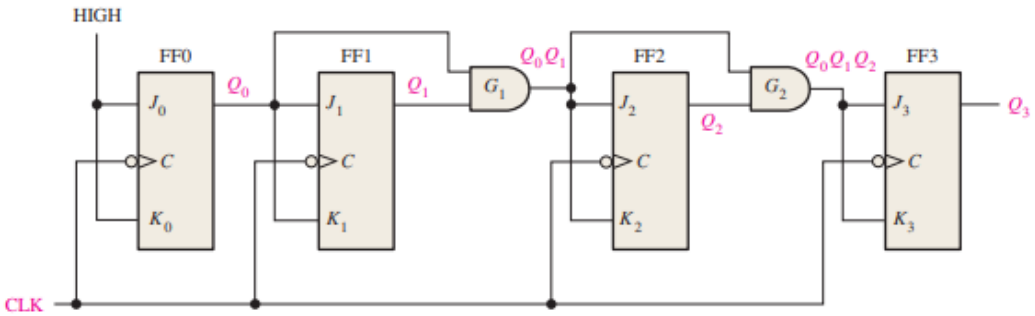
Clock Pulse	Q_2	Q_1	Q_0
Initially	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1
8 (recycles)	0	0	0

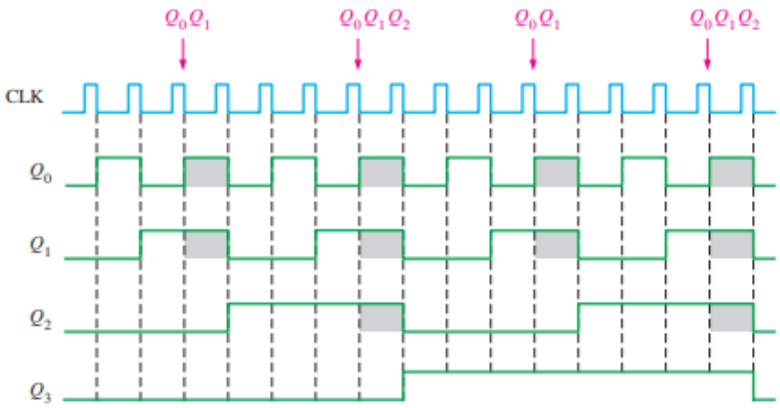
First, let's look at Q0. Notice that Q0 changes on each clock pulse as the counter progresses from its original state to its final state and then back to its original state. To produce this operation, FF0 must be held in the toggle mode by constant HIGHs on its J0 and K0 inputs. Notice that Q1 goes to the opposite state following each time Q0 is a 1. This change occurs at CLK2, CLK4, CLK6, and CLK8. The CLK8 pulse causes the counter to recycle. To produce this operation, Q0 is connected to the J1 and K1 inputs of FF1. When Q0 is a 1 and a clock pulse occurs, FF1 is in the toggle mode and therefore changes state. The other times, when Q0 is a 0, FF1 is in the no-change mode and remains in its present state. Next, let's see how FF2 is made to change at the proper times according to the binary sequence. Notice that both times Q2 changes state, it is preceded by the unique condition in which both Q0 and Q1 are HIGH. This condition is detected by the AND gate and applied to the J2 and K2 inputs of FF2. Whenever both Q0 and Q1 are HIGH, the output of the AND gate makes the J2 and K2 inputs of FF2 HIGH, and FF2 toggles on the following clock pulse. At all other times, the J2 and K2 inputs of FF2 are held LOW by the AND gate output, and FF2 does not change state.

Summary of the analysis of the counter in Figure 9–15.

	Outputs			*J-K* Inputs						At the Next Clock Pulse		
Clock Pulse	Q_2	Q_1	Q_0	J_2	K_2	J_1	K_1	J_0	K_0	**FF2**	**FF1**	**FF0**
Initially	0	0	0	0	0	0	0	1	1	NC*	NC	Toggle
1	0	0	1	0	0	1	1	1	1	NC	Toggle	Toggle
2	0	1	0	0	0	0	0	1	1	NC	NC	Toggle
3	0	1	1	1	1	1	1	1	1	Toggle	Toggle	Toggle
4	1	0	0	0	0	0	0	1	1	NC	NC	Toggle
5	1	0	1	0	0	1	1	1	1	NC	Toggle	Toggle
6	1	1	0	0	0	0	0	1	1	NC	NC	Toggle
7	1	1	1	1	1	1	1	1	1	Toggle	Toggle	Toggle
										Counter recycles back to 000.		

A 4-Bit Synchronous Binary Counter:





A counter is a sequential circuit that goes through a prescribed sequence of states upon the application of input pulses. The input pulses called count pulses, may be clock pulses, or they may originate from an external source and may occur at prescribed intervals of time or at random.

Synchronous Counter:

Synchronous counters are designed in such a way that the clock pulses are applied to the CP inputs of all the flip-flops. The common pulse triggers all the flip-flops simultaneously, rather than one at a time in succession.

In the 3-bit synchronous counter, we have used three j-k flip-flops. As in the diagram, The J and K inputs of FF0 are connected to HIGH. The inputs J and K of FF1 are connected to the output of FF0, and the J and K inputs of FF2 are connected to the output of an AND gate, which is fed by the outputs of FF0 and FF1.

Up/Down Synchronous Counters:

An up/down counter is one that is capable of progressing in either direction through a certain sequence. An up/down counter, sometimes called a bidirectional counter, can have any specified sequence of states. A 3-bit binary counter that advances upward through its sequence (0, 1, 2, 3, 4, 5, 6, 7) and then can be reversed so that it goes through the sequence in the opposite direction (7, 6, 5, 4, 3, 2, 1, 0) is an illustration of up/down sequential operation.

In general, most up/down counters can be reversed at any point in their sequence. For instance, the 3-bit binary counter can be made to go through the following sequence:

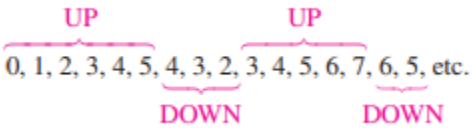


Table shows the complete up/down sequence for a 3-bit binary counter. The arrows indicate the state-to-state movement of the counter for both its UP and its DOWN modes of operation. An examination of Q0 for both the up and down sequences shows that FF0 toggles on each clock pulse. Thus, the J0 and K0 inputs of FF0 are

J0 = K0 = 1

Up/Down sequence for a 3-bit binary counter.

Clock Pulse	Up	Q_2	Q_1	Q_0	Down
0		0	0	0	
1		0	0	1	
2		0	1	0	
3		0	1	1	
4		1	0	0	
5		1	0	1	
6		1	1	0	
7		1	1	1	

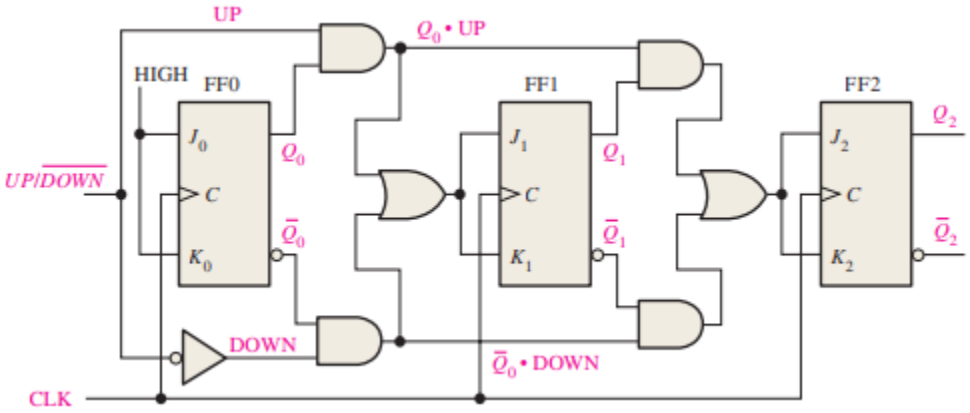
For the up sequence, Q1 changes state on the next clock pulse when Q0 = 1. For the down sequence, Q1 changes on the next clock pulse when Q0 = 0. Thus, the J1 and K1 inputs of FF1 must equal 1 under the conditions expressed by the following equation:

$$J_1 = K_1 = (Q_0 \cdot \text{UP}) + (\overline{Q}_0 \cdot \text{DOWN})$$

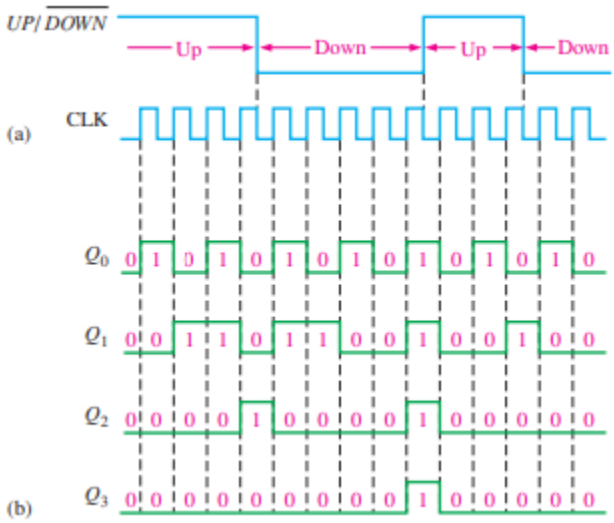
For the up sequence, Q2 changes state on the next clock pulse when Q0 = Q1 = 1. For the down sequence, Q2 changes on the next clock pulse when Q0 = Q1 = 0. Thus, the J2 and K2 inputs of FF2 must equal 1 under the conditions expressed by the following equation:

$$J_2 = K_2 = (Q_0 \cdot Q_1 \cdot \text{UP}) + (\overline{Q}_0 \cdot \overline{Q}_1 \cdot \text{DOWN})$$

Each of the conditions for the J and K inputs of each flip-flop produces a toggle at the appropriate point in the counter sequence.
Figure shows a basic implementation of a 3-bit up/down binary counter using the logic equations just developed for the J and K inputs of each flip-flop. Notice that the UP/DOWN control input is HIGH for UP and LOW for DOWN.



Eg:
Show the timing diagram and determine the sequence of a 4-bit synchronous binary up/down counter if the clock and UP/DOWN control inputs have waveforms as shown in Figure. The counter starts in the all-0s state and is positive edge-triggered.



The timing diagram showing the Q outputs is shown in Figure. From these waveforms, the counter sequence is as shown in Table.

Q_3	Q_2	Q_1	Q_0	
0	0	0	0	
0	0	0	1	
0	0	1	0	UP
0	0	1	1	
0	1	0	0	
0	0	1	1	
0	0	1	0	
0	0	0	1	DOWN
0	0	0	0	
1	1	1	1	
0	0	0	0	
0	0	0	1	UP
0	0	1	0	
0	0	0	1	
0	0	0	0	DOWN